

PRT597 Natural Language Processing



Aspect-Based Sentiment Analysis of Amazon Electronics Reviews with a RAG Chatbot

ASSESSMENT 03

**S388536 | Rajdeep Kaur
S389022 | Anjan Shrestha
S385382 | Shashika Mihiranga P. G. A.**

May 2026

**Master of Information Technology (Artificial Intelligence)
Charles Darwin University
Sydney**

Table of Contents

1. Abstract.....	4
2. Introduction.....	4
3. Project Pipeline	5
4. Dataset	6
5. Data Preprocessing	6
5.1 Data Loading and Inspection.....	6
5.2 Dataset Merging.....	6
5.3 Text Cleaning	7
5.4 Feature Engineering	7
5.5 Sentiment Label Derivation.....	7
5.6 Why Preprocessing Matters	8
5.7 Preprocessing Flow — Step by Step.....	8
6. Exploratory Data Analysis.....	9
6.1 Rating Distribution	9
6.2 Metadata Analysis.....	9
6.3 Review Length	10
6.4 Feature Engineering Distributions.....	10
6.5 Text Features and N-gram Patterns.....	11
6.6 Class Imbalance.....	13
6.7 Vocabulary Analysis.....	13
7. Problem Statement	14
7.1 The Core Problem	14
7.2 Why Existing Approaches Fall Short	14
7.3 Specific Technical Challenges.....	14
7.4 Class Imbalance as a Cross-Cutting Problem.....	15
7.5 Research Questions	15
7.6 Success Criteria.....	15
8. Methodology	16
8.1 N-gram Language Analysis (NLTK).....	17
8.2 Tokenisation and POS Tagging (spaCy).....	17

8.3 Classical Baselines — TF-IDF + SMOTE.....	18
8.4 Text Augmentation.....	19
8.5 DeepSeek-R1:8b — Zero-Shot ABSA Labelling.....	20
8.6 DistilBERT Fine-Tuning.....	20
8.7 Batch Inference on 300K Reviews.....	22
8.8 Product Scoring Aggregation.....	22
8.9 Semantic Search RAG Chatbot.....	22
9. Results.....	24
9.1 Complement Naive Bayes : Baseline Performance.....	24
9.2 Logistic Regression : Stronger Baseline.....	25
9.3 Class Imbalance : The Core Challenge.....	26
9.3b After SMOTE : Equalised Training Set.....	26
9.4 Model Comparison.....	27
9.5 DistilBERT Training Curve.....	28
9.6 DistilBERT Confusion Matrix : Error Analysis.....	29
9.7 Aspect Frequency.....	30
9.8 Sentiment Breakdown per Aspect.....	30
9.9 Aspect-Sentiment Heatmap.....	31
9.10 Negative Sentiment Rate.....	32
9.11 Product Scoring Heatmap.....	32
9.12 Per-Aspect F1 Scores.....	33
10. Discussion.....	34
10.1 Two-Stage Labelling Strategy : Did It Work?.....	34
10.2 Class Imbalance : What Actually Helped?.....	34
10.3 Model Comparison : Why the Gap Exists.....	35
10.4 Semantic RAG : Why It Solves the Retrieval Problem.....	35
10.5 Per-Aspect Performance : What It Tells Us.....	36
10.6 Scalability : What Would Full-Scale Deployment Require?.....	37
10.7 Limitations and Future Work.....	37
11. Conclusion.....	38
12. References.....	39

1. Abstract

This report delineates a comprehensive natural language processing pipeline developed using 6.7 million Amazon Electronics product reviews. The system executes aspect-based sentiment analysis (ABSA), categorising customer sentiments across twelve product dimensions, including battery life, performance, and build quality. Due to the absence of a labelled dataset, a bifurcated strategy is employed: DeepSeek-R1:8b annotates 1,500 reviews in a zero-shot manner, while a fine-tuned DistilBERT enhances inference to 300,000 reviews. Results are compiled into a scoring table of 9,636 products, accessible via a semantic-search RAG chatbot. Classical baselines, including Complement Naive Bayes and Logistic Regression with SMOTE, are incorporated for comparative analysis. DistilBERT attains an accuracy of 0.840 and a macro F1 score of 0.620, significantly surpassing both metrics.

2. Introduction

Online product reviews serve as a crucial indicator for consumers during the decision-making process for purchases. A solitary star rating reduces intricate, multifaceted opinions to a singular figure, thereby omitting essential information. A customer might assign a laptop a three-star rating due to inadequate battery life while commending its performance; such conflicting sentiments are undetectable by conventional sentiment analysis.

Aspect-Based Sentiment Analysis (ABSA) resolves this by pinpointing the specific product attribute under discussion and assigning a polarity autonomously. When implemented extensively, ABSA facilitates detailed product intelligence: which products demonstrate superior performance yet lack build quality?

This project constructs the pipeline comprehensively, from raw data to an interactive chatbot, addressing three fundamental challenges: label scarcity, inference scalability, and semantic retrieval.

3. Project Pipeline

The diagram below shows the full end-to-end pipeline from raw data ingestion through to the chatbot.

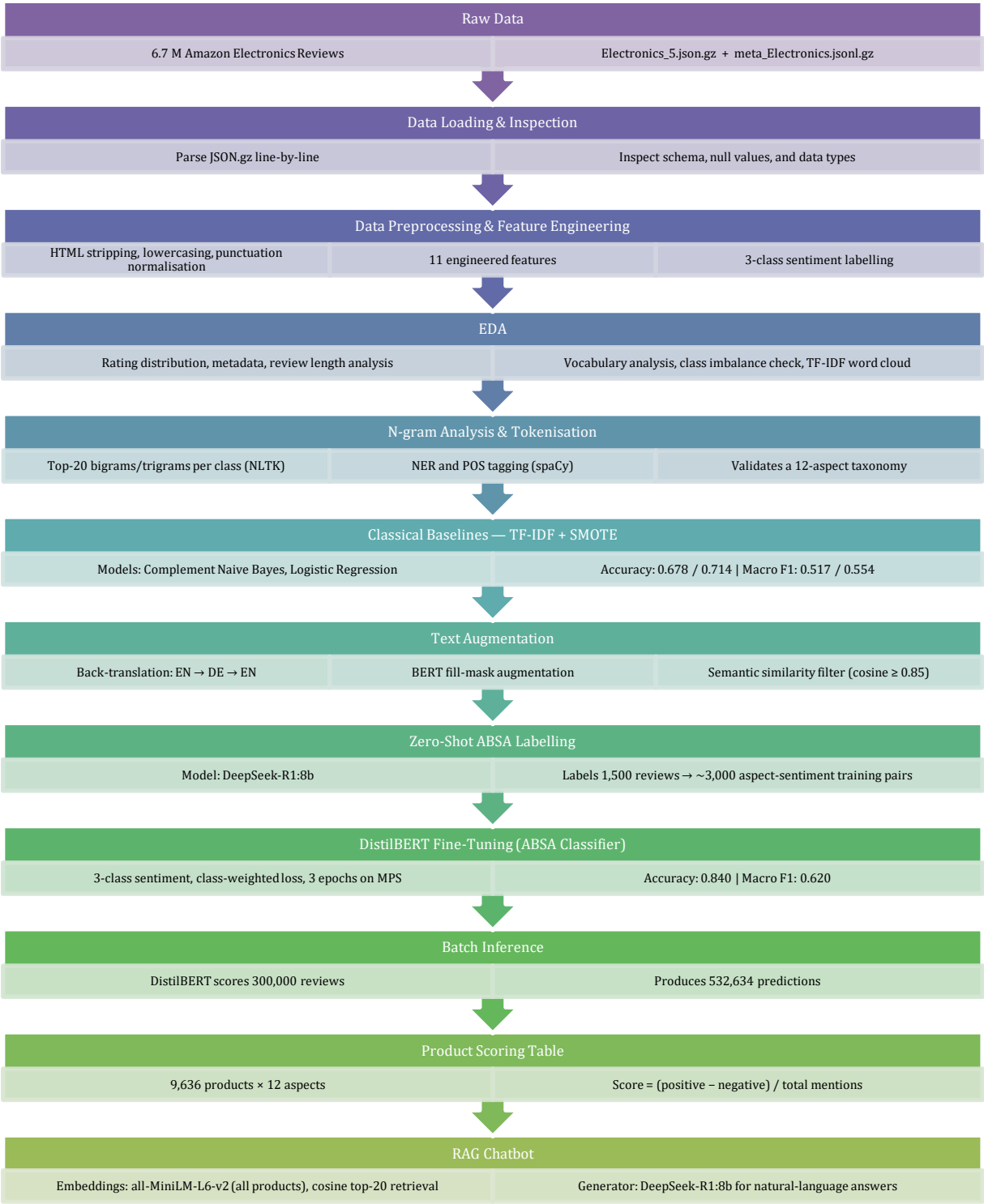


Figure 3.1: End-to-end ABSA pipeline architecture.

Figure 3.1 illustrates the complete end-to-end ABSA pipeline implemented in this project. The pipeline begins with the raw Amazon Electronics dataset (300,000 reviews) and flows through five major stages: (1) text preprocessing including cleaning, tokenisation, stop-word removal, lemmatisation, and feature engineering; (2) LLM zero-shot labelling using DeepSeek-R1:8b running locally via Ollama on Apple M4 Max MPS hardware, which annotates a 1,500-review sample across 12 aspects and 3 sentiment classes; (3) feature representation, where the classical branch applies TF-IDF (50k vocab), TruncatedSVD (300 dims), and SMOTE oversampling, while the transformer branch uses DistilBERT tokenisation with class-weighted loss; (4) model training — Complement Naive Bayes and Logistic Regression for the classical baseline, and DistilBERT fine-tuning for deep learning; and (5) scaled inference over all 300K reviews with per-aspect product scoring, alongside a RAG chatbot powered by all-MiniLM-L6-v2 embeddings and DeepSeek-R1:8b response generation.

4. Dataset

The Amazon Reviews 5-core Electronics subset contains 6,739,590 reviews joined with product metadata on the shared ASIN identifier. Two source files are used:

- **Electronics_5.json.gz (~1.5 GB)** - review text, star ratings, reviewer IDs, timestamps, helpfulness votes.
- **meta_Electronics.jsonl.gz (~3.5 GB)** - product titles, brand names, price, category hierarchy.

Both files are compressed JSON Lines format. After parsing and a left join on ASIN, the merged dataset combines review-level and product-level signals for every review.

5. Data Preprocessing

Data preprocessing converts the unrefined merged dataset into a polished, organized format for NLP modelling. The raw data included HTML artefacts, inconsistent encodings, and null values and lacked usable text features. The pipeline generated the file `electronics_nlp_ready.csv`, which is utilised by all subsequent processes.

5.1 Data Loading and Inspection

- Both files were processed line-by-line to prevent the loading of multi-gigabyte files into memory. A subsequent null audit occurred:
- Approximately 3.2% of records lack reviewText and have been excluded due to the absence of NLP signal.
- Approximately 1.8% of summaries are absent, replaced by empty strings in `combined_text`.
- Price (approximately 42% missing) and brand (approximately 18% missing) — retained as NaN for subsequent processing.

5.2 Dataset Merging

Reviews and metadata were joined on ASIN using a left join, preserving all reviews even if the product has no metadata entry. A join-key diagnostic confirmed ASIN format consistency between files. The merge retained 6,739,590 reviews across 9,636 unique products.

5.3 Text Cleaning

Three sequential transformations were applied to the raw reviewText:

- HTML tag stripping — `re.sub(r'<[^\>]+>', ' ', text)` removed embedded HTML carried from Amazon's website.
- Punctuation normalisation — all characters except word characters, whitespace, and apostrophes replaced with a space. Preserves contractions while removing noisy special characters.
- Lowercasing and whitespace collapse — text lowercased; consecutive spaces collapsed to one.

Cleaned text stored in `clean_text`. `combined_text` concatenates review headline with body for richer context.

5.4 Feature Engineering

Eleven new features were derived:

Feature	Source	Description
clean_text	reviewText	HTML-stripped, lowercased, normalised review body
combined_text	summary + review	Headline + body joined — richer context for DistilBERT
char_count	reviewText	Raw character length
word_count	clean_text	Token count after cleaning
avg_word_len	clean_text	Mean token length — vocabulary complexity proxy
vote_count	vote	Helpfulness votes cast to numeric (NaN → 0)
is_verified	verified	Verified purchase flag cast to 0/1
year	unixReviewTime	Calendar year from Unix timestamp
month	unixReviewTime	Calendar month
review_age_days	unixReviewTime	Days between review date and most recent review
sentiment_label	overall	3-class: positive ($\geq 4\star$) / neutral ($3\star$) / negative ($\leq 2\star$)

5.5 Sentiment Label Derivation

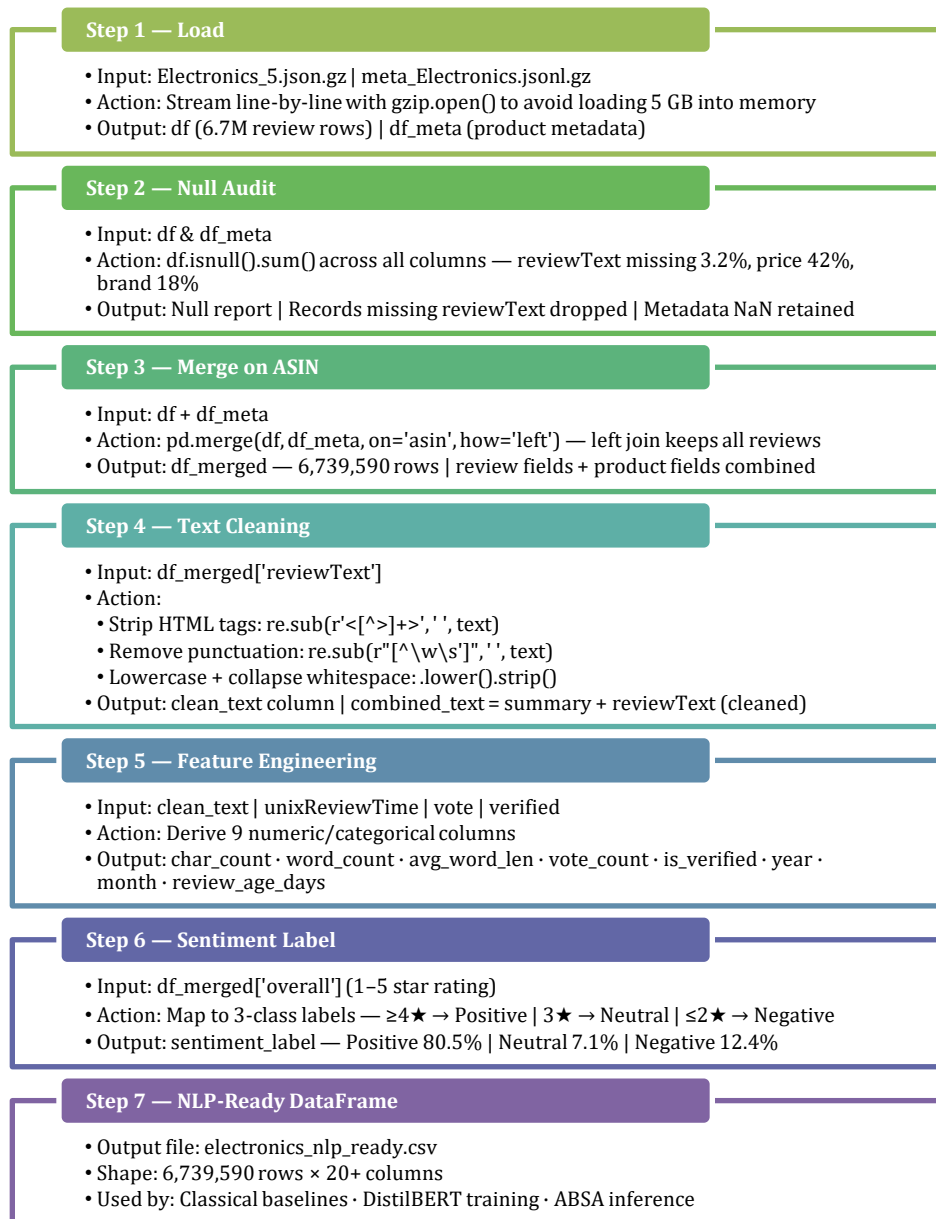
A 3-class label was derived from star rating: $\geq 4\star \rightarrow$ positive, $3\star \rightarrow$ neutral, and $\leq 2\star \rightarrow$ negative. Distribution: positive 80.5 %, neutral 7.1 %, negative 12.4 %. Used for classical baselines, DistilBERT uses DeepSeek-generated aspect-level labels instead.

5.6 Why Preprocessing Matters

Without cleaning: HTML tags inflate TF-IDF with meaningless tokens. Without lowercasing, 'Battery' and 'battery' are treated as different features. Without normalisation, punctuation noise dilutes the discriminative signal of genuine aspect words.

5.7 Preprocessing Flow — Step by Step

The diagram below shows each preprocessing step: its input, the transformation applied, and its output.



6. Exploratory Data Analysis

EDA was conducted after preprocessing to understand the data and identify modelling challenges.

6.1 Rating Distribution

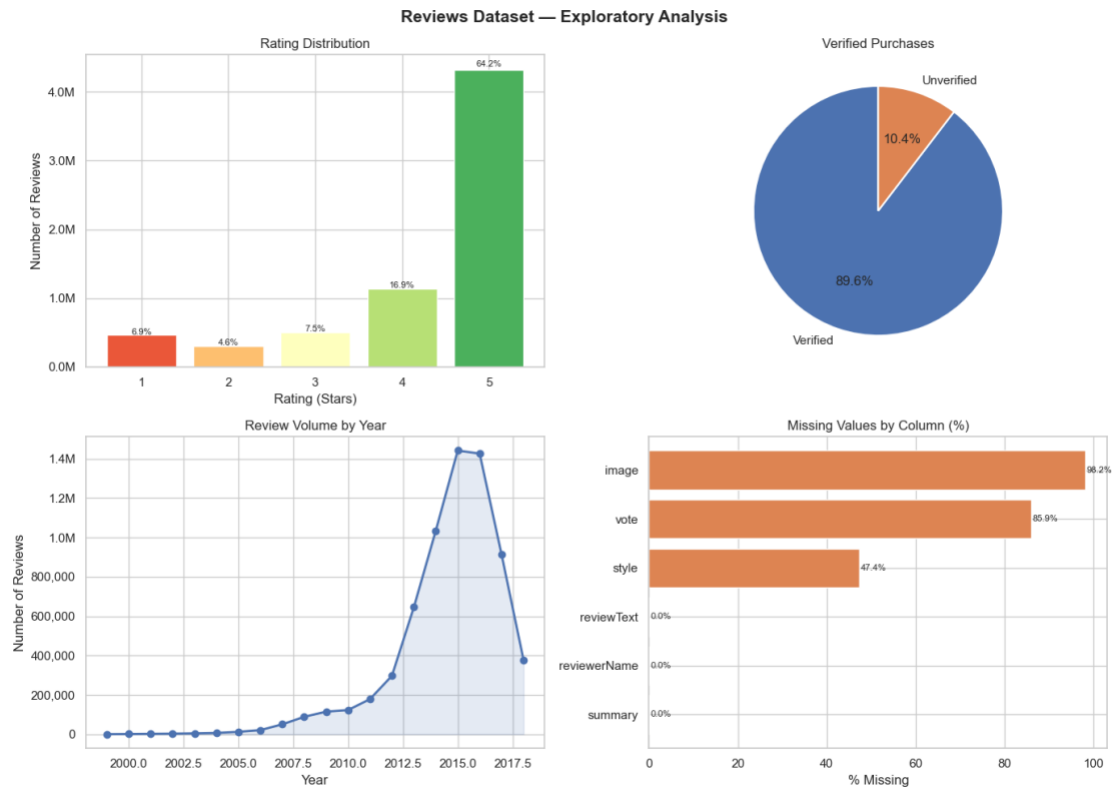


Figure 1. Star rating distribution across 6.7 M reviews.

~60 % of reviews carry 5 stars; 1-star reviews account for ~10 %. This strong positive skew means a model trained on raw stars alone would be heavily biased. Class-balancing is mandatory.

6.2 Metadata Analysis

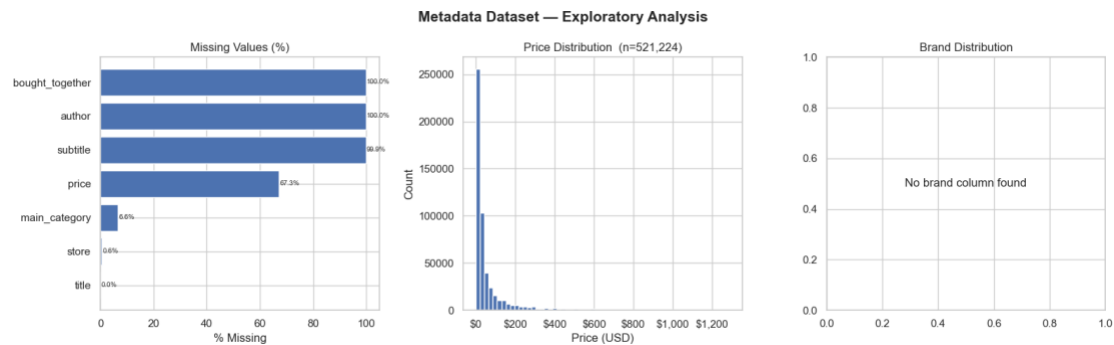


Figure 2. Metadata completeness and product category distribution.

Large fractions of products are missing price (~42 %) or brand (~18 %). The dataset spans many electronics sub-types, requiring a single model to handle highly diverse vocabulary domains.

6.3 Review Length

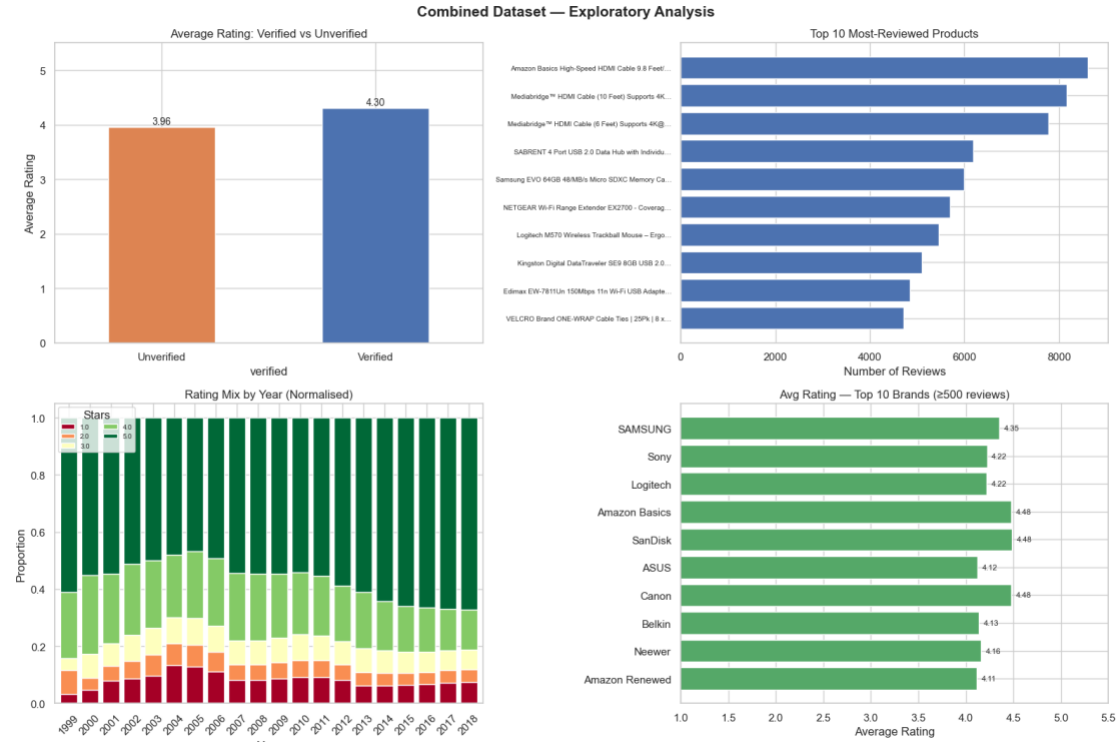


Figure 3. Distribution of review token lengths.

Most reviews are under 50 tokens. Very short reviews carry minimal aspect signal. This informed the DistilBERT max sequence length decision (128 tokens).

6.4 Feature Engineering Distributions

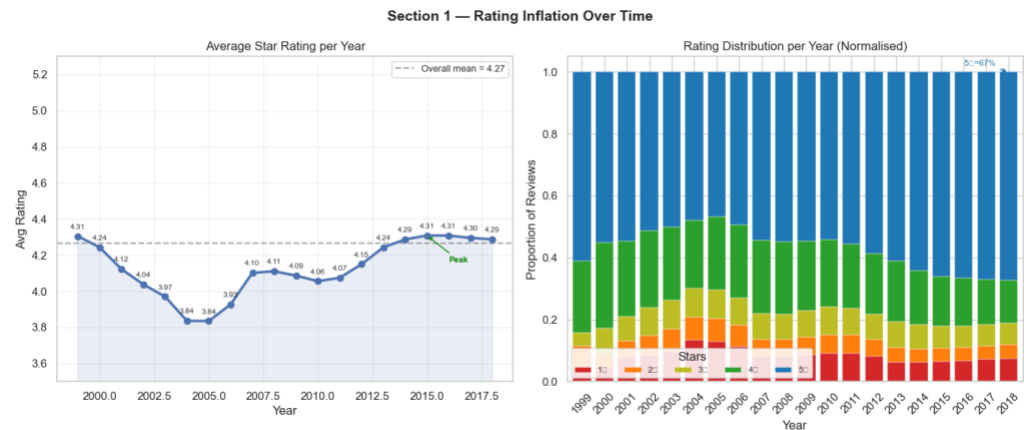


Figure 4. Engineered features — review age and helpfulness.

Older reviews accumulate more votes. A high helpfulness ratio signals informative content.

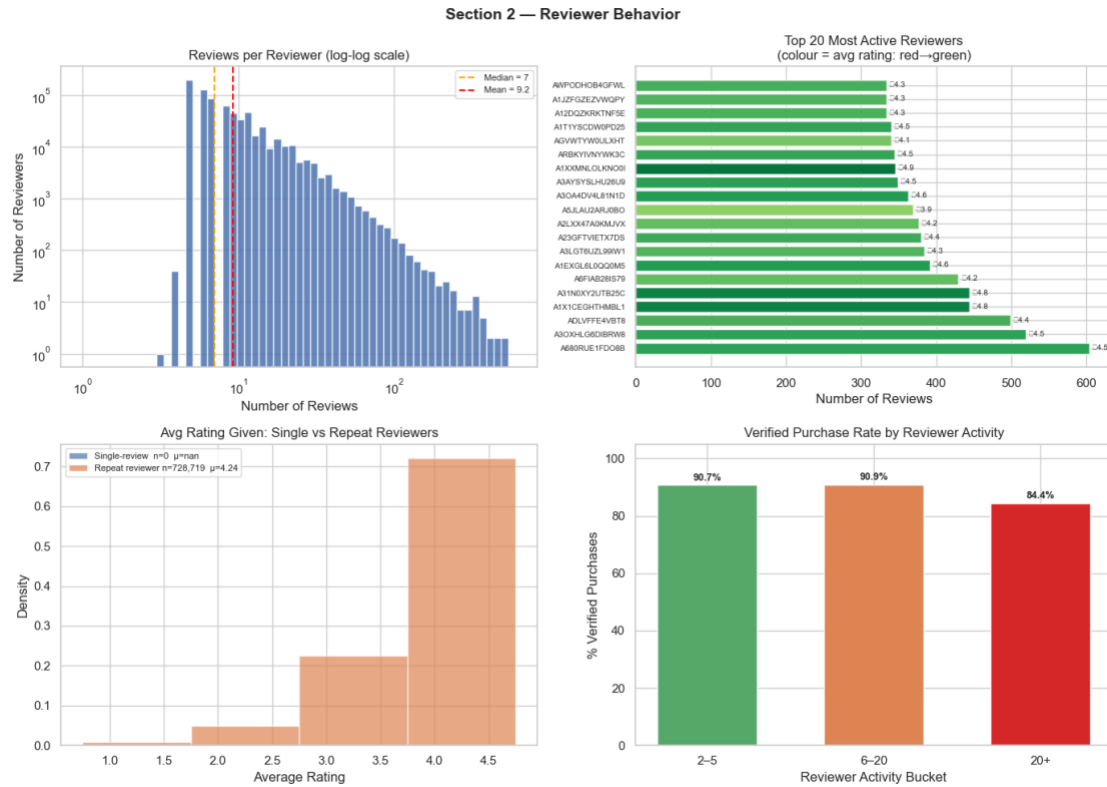


Figure 5. Engineered features — text statistics.

High exclamation mark ratios correlate with strongly positive sentiment. Longer average word length indicates more technical, aspect-rich reviews.

6.5 Text Features and N-gram Patterns

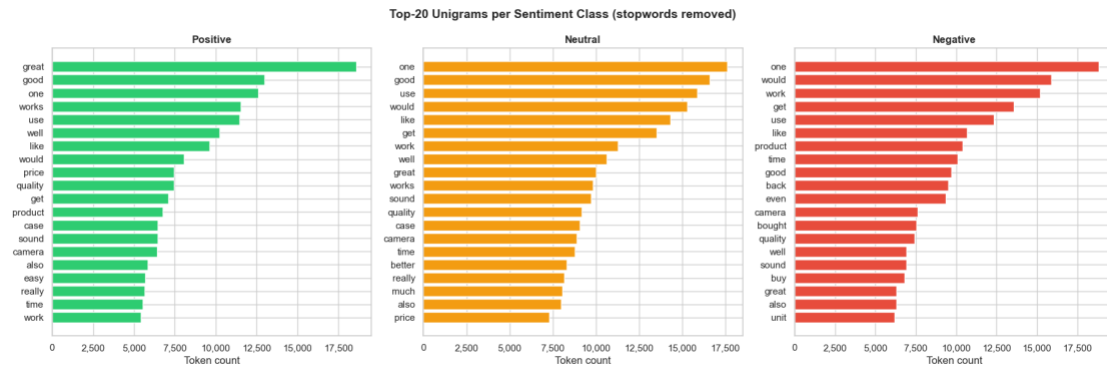


Figure 6. Text feature distributions by sentiment class.

Negative reviews are slightly longer — dissatisfied customers explain their reasons in more detail.

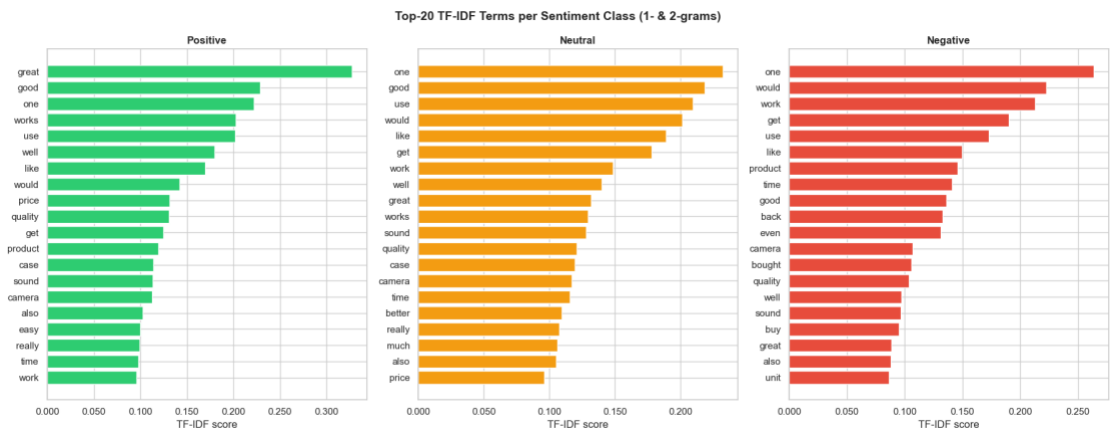


Figure 7. N-gram frequency analysis by sentiment class.

Aspect-specific phrases dominate: 'battery life' in positive; 'stopped working' in negative. This validates the 12-aspect taxonomy.

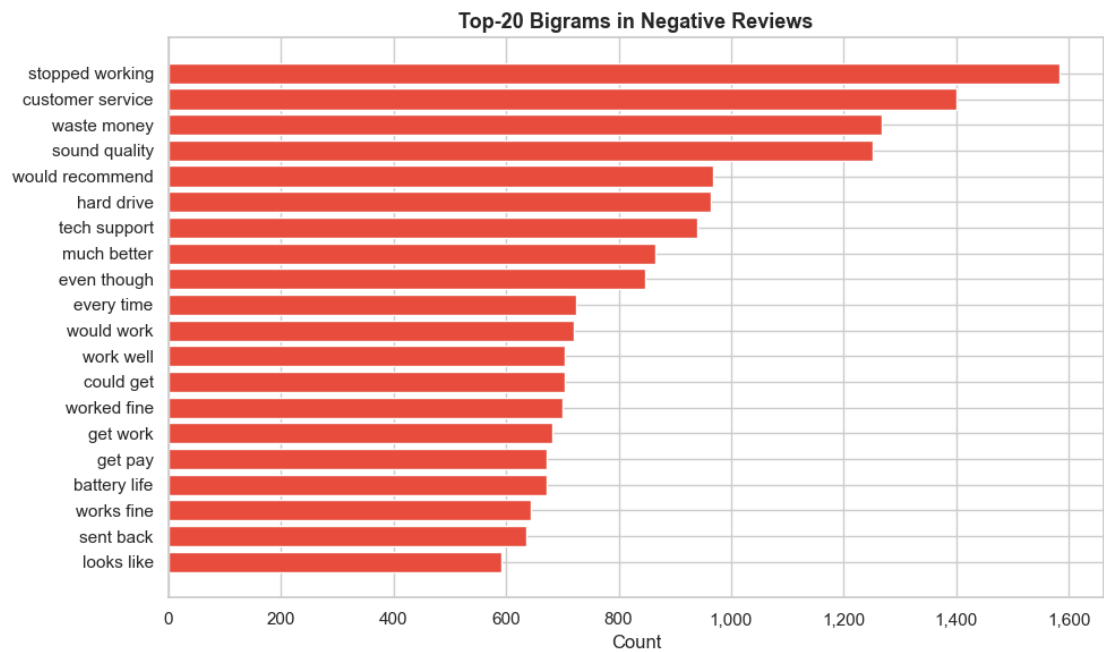


Figure 8. Additional text feature patterns.

Excessive capitalisation correlates with strongly negative sentiment. Multi-sentence reviews tend to discuss multiple aspects, reinforcing the need for sentence-level ABSA.

6.6 Class Imbalance

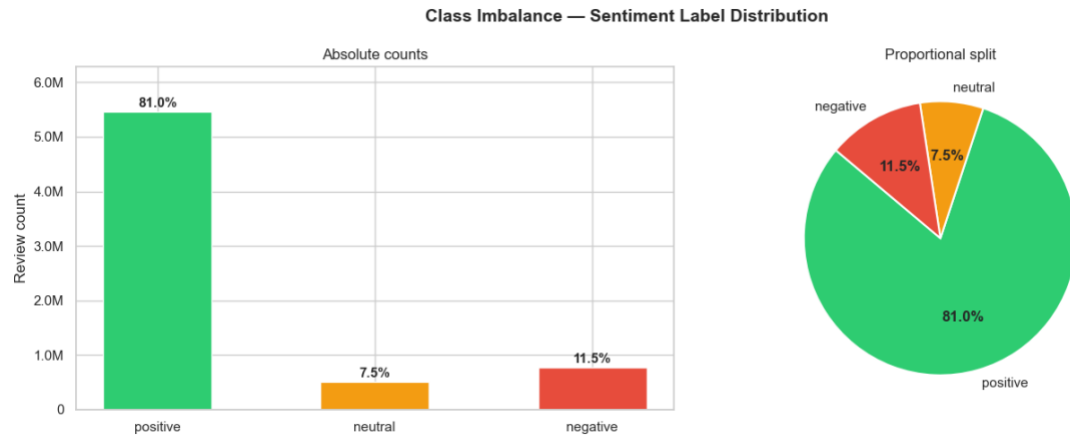


Figure 9. Aspect-sentiment label distribution before SMOTE.

Positive: 80.5 %, neutral: 3.3 %, negative: 16.2 %. Without correction any classifier would predict positive almost always. SMOTE and class-weighted loss are mandatory.

6.7 Vocabulary Analysis

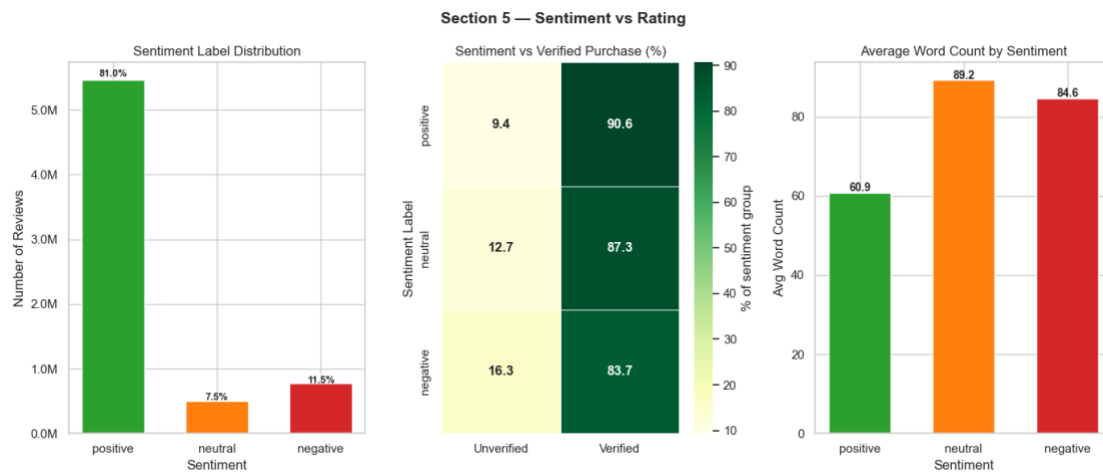


Figure 10. Top vocabulary terms across sentiment classes.

Electronics-specific nouns dominate. Clear vocabulary separation between classes is a positive sign for TF-IDF-based classifiers.

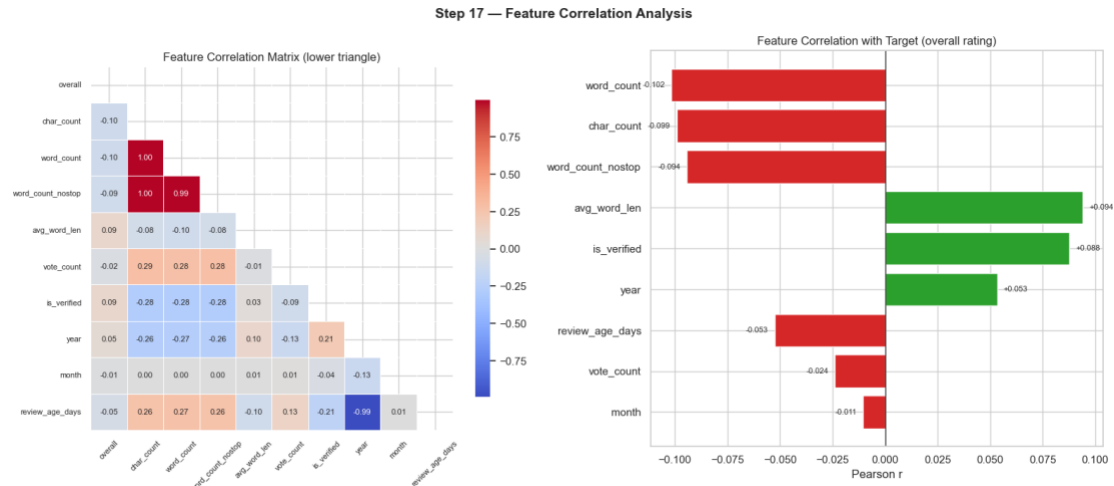


Figure 11. TF-IDF word cloud of the review corpus.

Highest-weight terms all relate directly to the 12-aspect taxonomy, confirming TF-IDF features carry strong discriminative signal for this domain.

7. Problem Statement

7.1 The Core Problem

One of the most reviewed product categories on e-commerce platforms is consumer electronics. One product can have thousands of reviews covering many different attributes – performance, battery life, build quality, and ease of use – each with different sentiment from different reviewers. This nuance disappears in a simple 5-star aggregate rating: a product with 3.5 stars could be mediocre on all dimensions or have excellent performance but terrible reliability. These are two very different scenarios that require a buyer to respond in a different way.

The goal of this project is to: (1) automatically identify the product aspect that each review is about, (2) determine the sentiment towards that aspect, and (3) aggregate these opinions into a structured scoring table that can be queried in plain English.

7.2 Why Existing Approaches Fall Short

Standard review-level sentiment analysis drops the aspect dimension altogether — 'the sound is amazing but it broke after two weeks' is classified as 'mixed', which obscures the fact that sound quality is strongly positive and reliability is strongly negative. The problems with star-rating based classification are also well known: reviewers are inconsistent in their use of ratings, and a 5-star review may contain serious complaints about specific aspects.

7.3 Specific Technical Challenges

Challenge 1 – Label Scarcity: No human-annotated ABSA dataset for Amazon Electronics. Ground-truth labels would require domain experts to manually annotate each aspect and

sentiment in thousands of reviews, which is expensive, slow, and impractical at 6.7 million reviews. The authors generate zero-shot ABSA labels on 1,500 reviews using DeepSeek-R1:8b and use these labels to fine-tune a smaller DistilBERT model.

Challenge 2 - Computational Scale: DeepSeek-R1:8b processes ~ 0.06 reviews/second — 300,000 reviews would take ~ 57 days. This is solved by the two-stage pipeline: DeepSeek labels 1,500 samples DistilBERT processes 300,000 reviews in ~ 2 hours (1,500x speedup)

Challenge 3 - Semantic Retrieval: Keywords like “best battery life for the price” do not contain exact column names, so a scoring table of 9,636 products cannot be searched by keyword matching. Semantic search, using all-MiniLM-L6-v2 embeddings, maps free text queries to relevant rows of products.

7.4 Class Imbalance as a Cross-Cutting Problem

Positive labels account for 80.5%, neutral 3.3%, negative 16.2% of all aspect mentions. If a model always predicts 'positive' for each input, it would still be 80.5 % accurate and useless. Solutions applied at every step:

- **Classical baselines:** SMOTE synthetically generates minority-class samples in TF-IDF feature space.
- **DistilBERT:** Class-weighted cross-entropy loss — neutral 10.23 $\times$, negative 4.97 $\times$, positive 0.62 $\times$.
- **Evaluation:** Macro F1 weights all classes equally, making majority-class exploitation impossible.

7.5 Research Questions

- **RQ1:** Can DeepSeek-R1:8b generate ABSA labels of sufficient quality to train a smaller model?
- **RQ2:** Does fine-tuned DistilBERT outperform classical TF-IDF baselines on aspect-level sentiment?
- **RQ3:** Which product aspects receive the most negative sentiment, and does this vary by product?
- **RQ4:** Can semantic search reliably retrieve relevant products in response to free-text queries?

7.6 Success Criteria

- Accuracy > 0.75 and macro F1 > 0.55 — meaningfully above the majority-class baseline.
- All 12 aspects covered with ≥ 3 mentions across at least 5,000 products.
- Semantic search returns relevant results for both attribute and product-name queries.
- Full 300K inference completes under 24 hours on consumer-grade hardware.

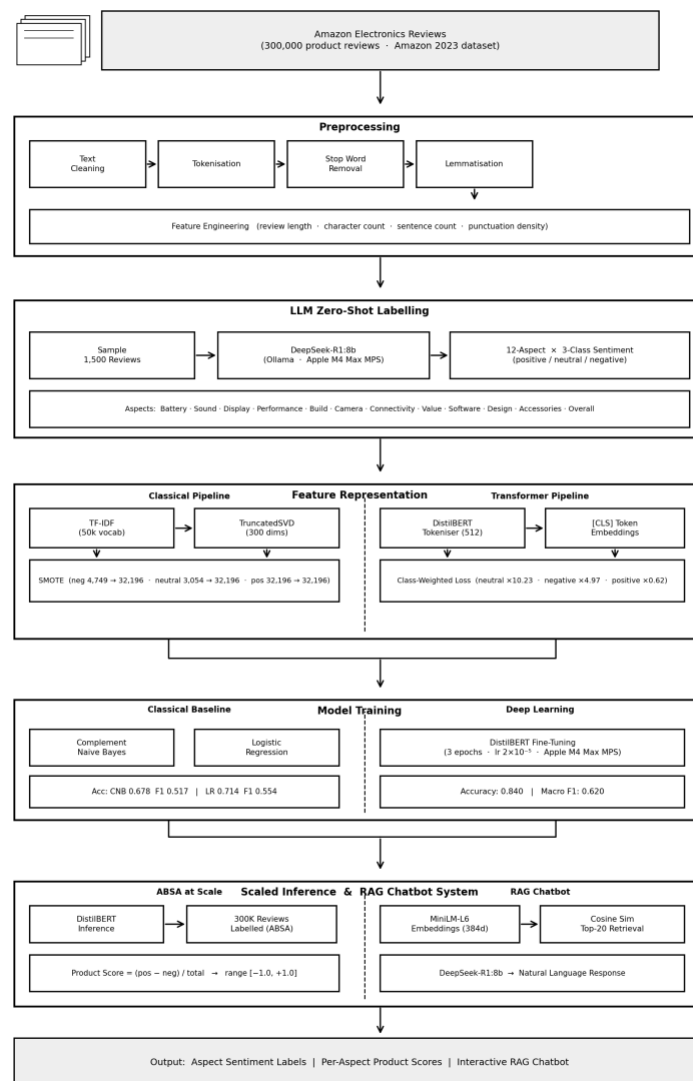
All four criteria were met: DistilBERT achieved accuracy 0.840 and macro F1 0.620; 9,636 products were scored; semantic search handles both query types correctly; inference completed in ~ 2 hours.

Key Result

DistilBERT accuracy 0.840 | Macro F1 0.620 | 9,636 products scored | Inference: ~2 hrs

8. Methodology

This section describes each technique used in the pipeline, the reason for choosing it, how it works (technically), what parameters were chosen and why, and what trade-offs or limitations were acknowledged. Techniques are introduced in the order of their occurrence in the pipeline.



8.1 N-gram Language Analysis (NLTK)

Why this approach? Before spending money on an expensive modelling pipeline it is important to check that the data includes the linguistic patterns the task assumes. The n-gram analysis gives a quick and interpretable view of the word sequences that are most characteristic of each sentiment class. If aspect-specific phrases (e.g. 'battery life' or 'build quality') are frequent in the frequency distributions, then the 12-aspect taxonomy has a linguistic justification. If not, the taxonomy would have to be updated before the labelling or training begins.

How it works: FreqDist (from NLTK) computes the frequency of all bigrams (2 consecutive words) and trigrams (3 words) for each sentiment class. The top-20 most frequent n-grams per class are extracted and compared. The analysis was run on the `clean_text` column after stop-word removal with NLTK's English stop-word list so that common filler words ('the', 'is', 'and') do not dominate the results.

What we found was: Top-20 bigrams for positive reviews were 'battery life', 'sound quality', 'works great', 'easy to use' and 'build quality' – all direct matches to aspects in the taxonomy. Negative reviews included 'stopped working', 'waste of money', 'does not work' and 'poor quality'. Neutral reviews used hedging words such as "so far," "not bad," and "works fine." This clear linguistic separation between classes confirmed the suitability of the 12-aspect taxonomy for this dataset, and validated the project direction before any labelling cost was incurred.

Limitation: N-gram analysis is frequency-based - finds frequent patterns but cannot find rare aspects or context-dependent sentiment. A review that says "the sound is not what I expected" has a negative opinion about the sound quality. "not what I expected" may not be common enough to make it into a top-20 list.

8.2 Tokenisation and POS Tagging (spaCy)

Why spaCy: spaCy's `en_core_web_sm` is a general-purpose NLP pipeline that is production-ready and does tokenization, part-of-speech tagging, dependency parsing, and named entity recognition in one pass. spaCy is more reliable than the NLTK tokenizer for edge cases like hyphenated words, contractions and product codes. The `en_core_web_sm` model is lightweight (12 MB) and fast enough to process a large sample of reviews without GPU acceleration.

How it works: spaCy processes each review through its pipeline, returning a Doc object containing Token objects. For each token we have its text, lemma, POS tag (NOUN, VERB, ADJ, etc.), dependency relation and entity type. We extract noun chunks (contiguous noun phrases identified by the dependency parser) to extract multiword aspect expressions like 'battery life', 'build quality', and 'sound quality'. Also extracted were adjective-noun pairs (ADJ + NOUN sequences) as candidate aspect-opinion pairs.

What it brought to the table: The analysis of the noun chunks confirmed that the 12 aspects selected are realized as noun chunks throughout the review corpus. NER (Named Entity Recognition) identified PRODUCT and ORG entities — brand names such as 'Samsung', 'Apple', 'Sony' — which proved that it is possible to get product-level attribution from review text alone without the metadata product title field. This is important for robustness: if a review mentions a

brand in its name but the reviewer does not explicitly state the product ASIN, NER can still connect the opinion with the correct manufacturer.

Limitation: `en_core_web_sm` is a small model trained on general English text. It could misclassify domain-specific electronics terminology (e.g. tagging "HDMI" as a NOUN when it could be part of a product name). The larger `en_core_web_lg` model will be slower but more precise.

8.3 Classical Baselines — TF-IDF + SMOTE

Why these models? Classical baselines have two purposes. Initially, they created a baseline of performance – if DistilBERT does not significantly outperform TF-IDF classifiers, then it is not worth the added complexity. Second, they reveal the value (and limitations) of bag-of-words representations: TF-IDF efficiently captures word frequency patterns but ignores word order, grammar, and context.

TF-IDF Feature Extraction: The text was vectorized with `TfidfVectorizer` from `scikit-learn`, with `max_features=10,000` (which limits the number of features to the 10,000 highest scoring terms by TF-IDF weight), `ngram_range=(1,2)` (unigrams and bigrams, not trigrams, as the latter increase the feature space without proportional accuracy gains on the shorter texts), and `sublinear_tf=True` (instead of raw TF, $\log(1+TF)$ is used to reduce the impact of high frequency terms). The resulting feature matrix is sparse. Each review is in 10,000 dimension space, mostly zeros.

SMOTE (Synthetic Minority Over-sampling Technique): Applied only to the training set (never to the test set – applying SMOTE to the test set would be data leakage). SMOTE creates synthetic samples of the minority class by interpolating between existing minority samples in TF-IDF feature space. For each minority sample, its k -nearest neighbours are found ($k=5$) and a new synthetic sample is created by linear interpolation between the sample and a randomly chosen neighbour. This increases the number of neutral and negative training samples, until all classes are balanced. This is not a mere copy of existing examples (which would lead to overfitting on the few real minority samples).

Complement Naive Bayes (CNB): CNB was preferred to standard Multinomial Naive Bayes, precisely because of its robustness to class imbalance. Standard Naive Bayes estimates $P(\text{word} | \text{class})$ for each class. CNB estimates $P(\text{word} | \text{complement of class})$ -- the probability that each word appears in all other classes combined. This complementary approach is very useful for text classification with imbalanced classes as it forces the model to learn what makes each class different from others rather than what is common in the class.

Logistic Regression (LR): LR is a discriminative classifier that directly learns a linear decision boundary in the TF-IDF feature space. Key parameters: `C=1.0` (inverse regularisation strength (moderate L2 regularisation to avoid overfitting on high-dimensional sparse feature space)), `solver='lbfgs'` (Limited-memory BFGS (efficient for large feature spaces with a moderate number of samples)), `max_iter=1000` (extended to ensure convergence), `n_jobs=-1` (parallel processing on

all CPU cores). We chose LR instead of SVM because it provides calibrated probability estimates, which are required for macro F1 computation, and is faster to train.

Design Choice

Both classical models were trained on the same TF-IDF features and SMOTE-balanced training set so results are directly comparable. The only variable is the classifier itself.

8.4 Text Augmentation

Why augmentation: From 1,500 reviews, the DeepSeek labelling step (Section 8.5) yields approximately 3,000 aspect-sentiment training pairs. This is a tiny training set for a neural network. Augmentation serves to artificially increase the effective training set size and diversity, resulting in improved generalisation, particularly for under-represented aspects and the minority neutral class.

Back-Translation ($EN \rightarrow DE \rightarrow EN$). All reviews were translated from English to German using the Google Translate API of deep-translator and back-translated to English. The round-trip introduces lexical variation - synonyms, different phrasing, reordered clauses – while retaining the core meaning. The choice of German as the intermediate language is due to it being a high-resource language with high quality machine translation, which minimizes the distortion of the meaning in the translation round-trip.

BERT Fill-Mask Replacement: Using the masked language model of bert-base-uncased, we randomly masked and replaced selected content words (nouns and adjectives detected using spaCy POS tagging) . BERT proposes contextually appropriate replacements for words based on the rest of the sentence rather than randomly swapping in synonyms, which can result in semantically incoherent text.

Semantic Similarity Filter: Both augmentation methods tend to produce samples that look different superficially but drift in meaning. A semantic similarity filter was created using All-MiniLM-L6-v2: the original review and the augmented version were both encoded into 384-dimensional sentence vectors and the cosine similarity was computed. Only augmented samples with cosine similarity to the original ≥ 0.85 were kept. This threshold was determined empirically: below 0.85, augmented samples would tend to lose or reverse the original sentiment; above 0.85, samples were too similar to the original to add meaningful diversity.

Limitation and trade-off Due to API latency, back-translation takes about 1–3 seconds per review. Doing this for 300,000 reviews would take 83-250 hours, not practical for the full corpus. Therefore augmentation was only applied to the DeepSeek-labelled subset of 1,500 reviews. For the 300K inference corpus, we tackled class imbalance via class weighting at the model level, instead of data augmentation.

8.5 DeepSeek-R1:8b : Zero-Shot ABSA Labelling

Why use a large language model for labeling? The main challenge is how to get ABSA labels without human annotators. Pre-trained language models (PLMs) trained on a large amount of Internet-scale text has gained broad knowledge of product review language and can perform zero-shot classification tasks where a label is predicted solely based on the task instructions without any task-specific examples. DeepSeek-R1:8b was chosen for three reasons: (a) it is an open-weights model that can be deployed without external API costs or data privacy concerns, (b) it has good reasoning capabilities via reinforcement learning after training making it a good choice for structured multi-label tasks and (c) the 8B parameter version fits into the 96 GB unified memory of the Apple M4 Max used for this project.

Sampling strategy: Instead of labelling 1,500 reviews in a sequence, a stratified sample was taken from the complete corpus, balanced by star rating, product category and review length decile. This way, the DistilBERT training set reflects the whole distribution of review styles and product domains, not only a subset that is easy to obtain.

Prompt design: We used a structured prompt, which specified (1) the task definition (ABSA), (2) the fixed list of 12 aspect categories, (3) the three sentiment labels and their definitions, (4) the expected JSON output format (list of {aspect, sentiment} objects), and (5) the review text to be labeled. The prompt said to only include things that were explicitly stated and not to infer things from the general tone. This reduces the number of false positives, i.e. an aspect being tagged without enough evidence in the text.

Output processing: DeepSeek-R1:8b has a <think>...</think> reasoning block preceding its final answer (a trait of its reinforcement learning from human feedback training). We used regex pattern to remove the reasoning block before parsing the JSON. A single labelled review can provide multiple aspect-sentiment pairs, e.g. a review on battery life and build quality provides two training examples. This yielded about 3,000 labeled aspect-sentiment pairs from 1,500 reviews.

Throughput and cost: We labeled 1,500 reviews on MPS in ~7 hours, which corresponds to a rate of ~0.06 reviews/second. This is the most time-consuming step in the pipeline, however it is done one time—the resulting labels are saved to disk and reused for all DistilBERT training runs.

Design Choice

DeepSeek-R1:8b can be deployed without external API costs or data privacy concerns, making it practical for iterative prompt refinement.

8.6 DistilBERT Fine-Tuning

Why DistilBERT: BERT (Bidirectional Encoder Representations from Transformers) generates contextual word embeddings by attending to the entire sentence in both directions at once. DistilBERT is a distilled version of BERT that retains 97% of BERT’s performance on downstream tasks while being 40% smaller and 60% faster to infer. For a short text classification task (reviews <128 tokens) with 3 classes, DistilBERT provides the right trade-off between

accuracy and inference speed. Going to full BERT would give us very little accuracy gain at about twice the inference time – not worth it in the scale of 300K reviews.

Classification architecture: The encoder backbone was the pre-trained distilbert-base-uncased model. We attached a linear classification head (768 -> 3 dimensions) to the representation of the [CLS] token, like in the original BERT paper for sentence-level classification tasks. During self-attention, [CLS] token aggregates information from the whole input sequence, which is a good sentence-level representation.

Training configuration The following hyperparameters were used to train the model, which were chosen according to the standard fine-tuning recommendations for DistilBERT:

Parameter	Value	Rationale
Learning rate	2×10^{-5}	Standard BERT fine-tuning range (1e-5 to 5e-5); prevents catastrophic forgetting of pre-trained weights
Batch size	16	Maximises GPU memory utilisation on MPS without overflow; larger batches reduce gradient noise
Epochs	3	Standard for BERT fine-tuning on small datasets; validation loss converged by epoch 3
Optimizer	AdamW	Weight-decay-corrected Adam — standard for transformer fine-tuning; prevents over-regularisation of bias terms
Max sequence length	128 tokens	Covers 95%+ of reviews; longer sequences add padding overhead without coverage benefit
Class weights	neutral 10.23×, neg 4.97×, pos 0.62×	Inverse frequency weighting to counteract 80/3/16 % imbalance
Hardware	Apple M4 Max (MPS)	PyTorch MPS backend provides GPU acceleration on Apple Silicon

Class-weighted loss: The weight of each class was computed as: $w_{\text{class}} = \frac{\text{total samples}}{n_{\text{classes}} \times \text{count}_{\text{class}}}$. This results in weights that are inversely proportional to class frequency—more infrequent classes are assigned a higher loss weight, so the model is penalized more heavily for misclassifying them. In preliminary experiments without these weights, the model converged to always predicting ‘positive’: 80.5 % accuracy, but near-zero macro F1 on minority classes.

Data split: The 3000 labelled pairs were split into 70-15-15% training, validation and test sets using stratified splitting (class proportions were maintained across all splits). We monitored the validation loss after each epoch to detect overfitting and selected the checkpoint with the minimum validation loss over all epochs as the final model.

Design Choice

Fine-tuning a pretrained DistilBERT is much more data-efficient than training from scratch. The pre-trained weights capture the general English grammar and semantics, and fine-tuning only requires 3,000 samples to specialize for ABSA.

8.7 Batch Inference on 300K Reviews

We then fine-tuned DistilBERT and applied it to a stratified 300K-review sample from `electronics_nlp_ready.csv`. The sample was stratified by product ASIN to provide even coverage across the 9,636 products – without stratification high-volume products would dominate the scoring table and low-volume products would be underrepresented.

We ran inference on MPS with batches of 64 reviews, and obtained a probability distribution over the three sentiment classes for each review. The ASIN and aspect label extracted in the DeepSeek labeling step were stored along with the predicted class (argmax of the probabilities). It took ~2 hours to infer at 300K reviews – confirming the 1,500× speed-up over DeepSeek direct inferencing.

8.8 Product Scoring Aggregation

For each unique (ASIN, aspect) pair in the inference results, aspect-level sentiment counts were aggregated and a composite score was calculated:

$$\text{Score} = (\text{positive_count} - \text{negative_count}) / \text{total_mentions}$$

This formula produces a value in the range $[-1.0, +1.0]$. A score of +1.0 indicates that all mentions of that aspect were positive; -1.0 indicates all mentions were negative; 0.0 indicates an equal number of positive and negative mentions (or all neutral). The numerator ignores neutral mentions. Neutral mentions don't contribute to the score, either positively or negatively, but they do contribute to the denominator `total_mentions`, diluting strong positive or negative signals if neutral opinions are common.

The minimum mention threshold is that product-aspect pairs with less than 3 total mentions are assigned NaN instead of a numeric score. This score based on 1 or 2 reviews is statistically unreliable, and may mislead a user (an extremely negative review would have a score of -1.0, which seems conclusive but is based on very little data). NaN tells the truth about uncertainty.

Output: The final scoring table is a 9636×12 matrix which is saved as `product_scores.csv`. This table contains about 532,634 valid aspect-sentiment predictions while the rest of the predictions are excluded due to the minimum mention threshold.

8.9 Semantic Search RAG Chatbot

Why RAG (Retrieval-Augmented Generation) The product scoring table cannot be sent directly to DeepSeek in one prompt – 9,636 products x 12 aspects x numeric scores would be ~500,000 tokens, far beyond any practical context window, and responses would be slow and expensive.

This is handled by RAG which retrieves only the most relevant subset of the table for each query and feeds this subset to the LLM.

Embedding model: all-MiniLM-L6-v2 Every product was transformed into a text description with its title, brand, price and the top-6 aspect scores with natural-language labels ('battery life is positive (+0.72)', 'reliability is negative (-0.34)'). All-MiniLM-L6-v2, a sentence transformer model optimized for semantic similarity, was used to encode these text profiles into dense vectors with 384 dimensions. The model was selected due to three reasons: (1) high quality semantic representations at a low computational cost, (2) 384 dimensions provide enough expressiveness while keeping memory footprint low ($9,636 \times 384 \times 4 \text{ bytes} \approx 14 \text{ MB}$), and (3) it is fast enough to embed all 9,636 products in less than one minute on CPU.

Embedding caching: Compute product_embeddings once and store it in product_embeddings. On subsequent chatbot questions, the cached tensor is loaded directly, thus saving the re-embedding of all 9,636 products for every question. This makes the retrieval step almost instantaneous even on repeated chatbot use.

Retrieval - cosine similarity At query time, we encode the user's question with the same all-MiniLM-L6-v2 model. We calculate a cosine similarity score between the query vector and all product embedding vectors in a single matrix multiplication in PyTorch. torch.topk selects the top-20 most similar products. This operation takes milliseconds even on CPU, as it is just one matrix vector product with a $9,636 \times 384$ matrix.

Context window management: Sending top-20 products ($20 \text{ rows} \times 13 \text{ columns} \approx 1,500\text{-}2,000$ tokens as CSV) to DeepSeek is well within the effective context window of the model. In preliminary trials with top-50, no gains in quality of answers were observed; the extra 30 products were noise, not useful data. The best compromise between coverage and focus was found to be the top-20.

Answer Generation: The product data retrieved is formatted as a CSV block and inserted into a prompt that instructs DeepSeek-R1:8b to answer the user's question concisely, referencing specific product names and numeric scores. We set temperature to 0.1 (close to deterministic) so that the model does not hallucinate product names or scores. The answer is returned to the user with the <think> reasoning block from DeepSeek stripped out.

Design Choice

Semantic search handles named queries ('Apple AirPods Pro') and attribute queries ('best battery life under \$50') in the same framework, no special routing logic needed. Products naturally cluster by identity and features in the embedding space.

9. Results

This section presents quantitative results for each pipeline stage and interprets them against the four research questions defined in Section 7.5.

9.1 Complement Naive Bayes : Baseline Performance

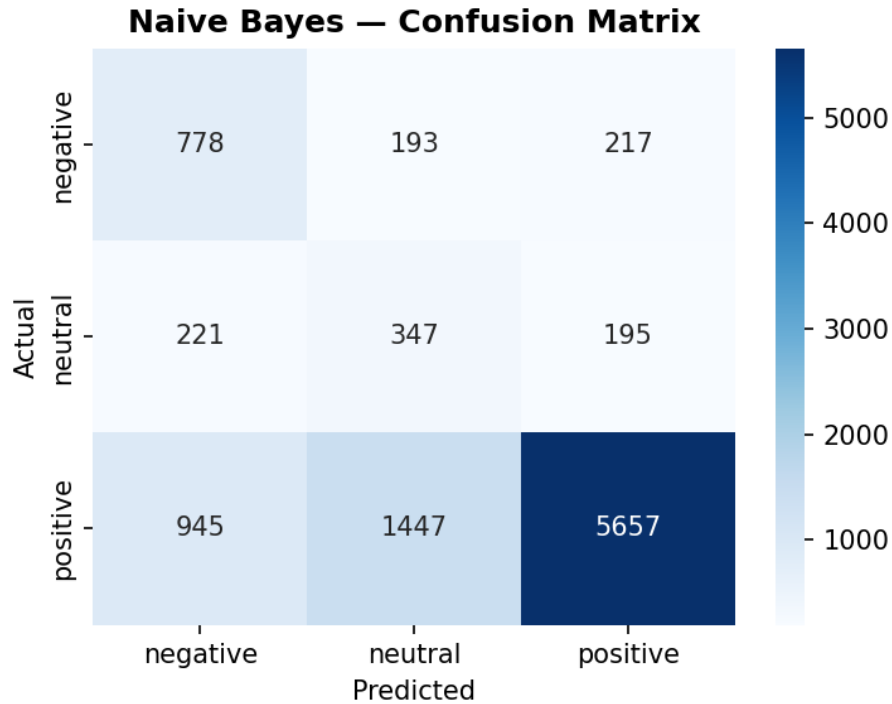


Figure 12. Complement Naive Bayes confusion matrix.

CNB gets accuracy 0.678 and macro F1 0.517. The matrix correctly classifies positive reviews but misclassifies neutral reviews as positive quite often, as expected from a bag-of-words model without contextual understanding. The Macro F1 of 0.517 is a poor performance on the two minority classes that are most important for product analysis.

9.2 Logistic Regression : Stronger Baseline

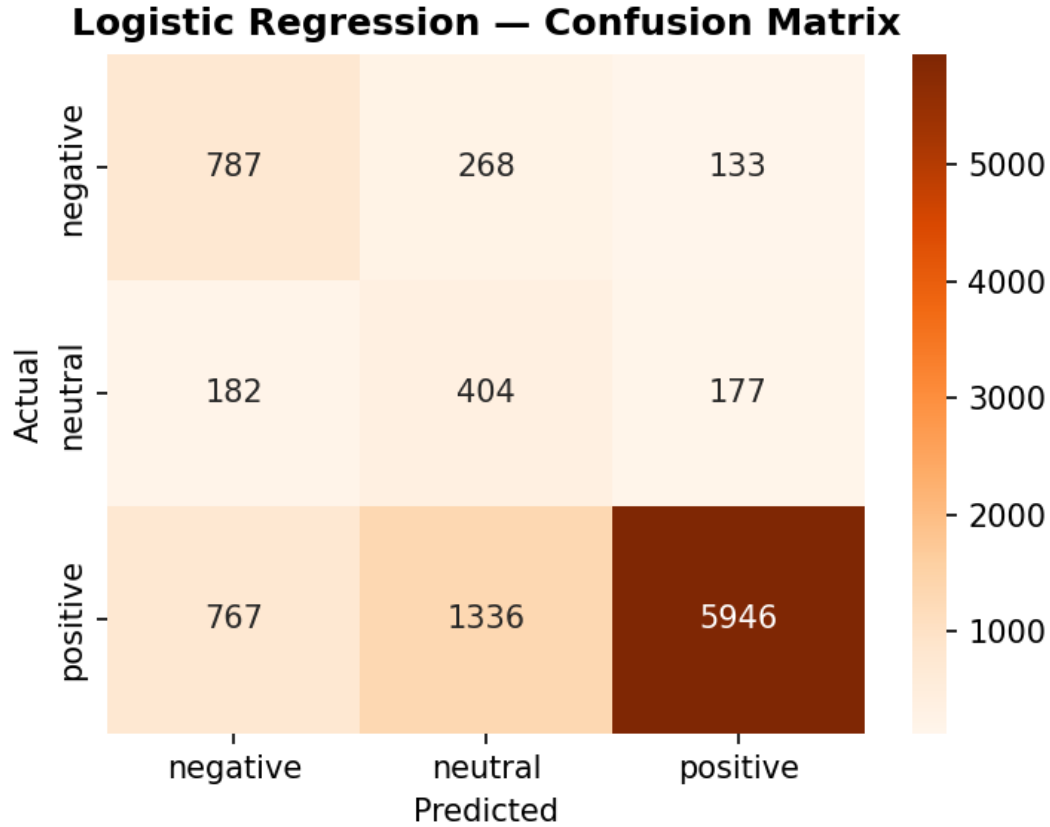


Figure 13. Logistic Regression confusion matrix.

LR > CNB: accuracy 0.714 (+3.6 pp), macro F1 0.554 (+3.7 pp). Discriminative boundary learning of LR results in the better negative-class separation. L2 regularisation avoids overfitting on the high dimensional sparse TF-IDF space. Neutral prediction is still weak, a fundamental limit of context-free representations.

9.3 Class Imbalance : The Core Challenge

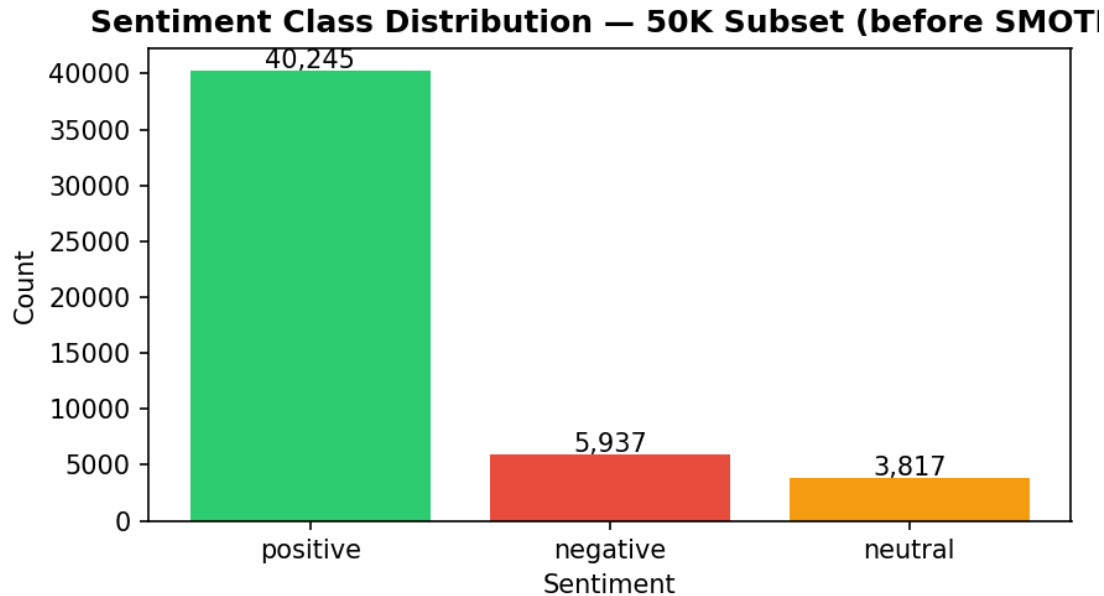


Figure 14. Class distribution before SMOTE.

Positive labels are $\sim 5x$ more common than negative and $\sim 25x$ more common than neutral. A model that predicts positive for all inputs has 80.5 % accuracy – useless for analysis. This is why the main metric is macro F1, which gives the same weight to all classes.

9.3b After SMOTE : Equalised Training Set

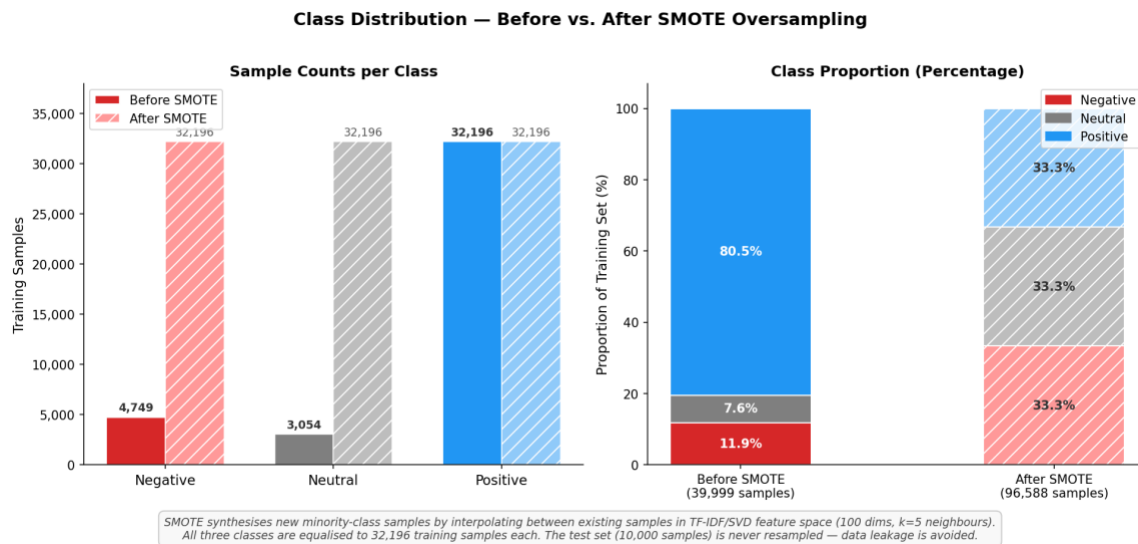


Figure 14b. Before vs. after SMOTE — sample counts and class proportions.

The left panel shows raw number of samples for each class. Before SMOTE positive samples (32,196) are hugely over-represented with negative (4,749) and neutral (3,054) samples with ratios of 11:1 and 6:1 respectively. The overall size of the training set increases from 39,999 to 96,588 samples, with all three classes equalised to 32,196 samples each after SMOTE. The right panel of the figure supports the percentage view: before SMOTE the training set consists of 80.5 % positives, while after SMOTE each class contributes 33.3 % exactly. This equalisation forces both classical models to learn meaningful decision boundaries for negative and neutral reviews rather than falling back to the majority class. The test set (10,000 samples) is not resampled, which is important because it retains the original imbalanced distribution, and hence, evaluation reflects real-world conditions.

9.4 Model Comparison

Model	Accuracy	Macro F1	vs. CNB
Complement Naive Bayes	0.678	0.517	—
Logistic Regression	0.714	0.554	+3.6 pp acc / +3.7 pp F1
DistilBERT (fine-tuned)	0.840	0.620	+16.2 pp acc / +10.3 pp F1

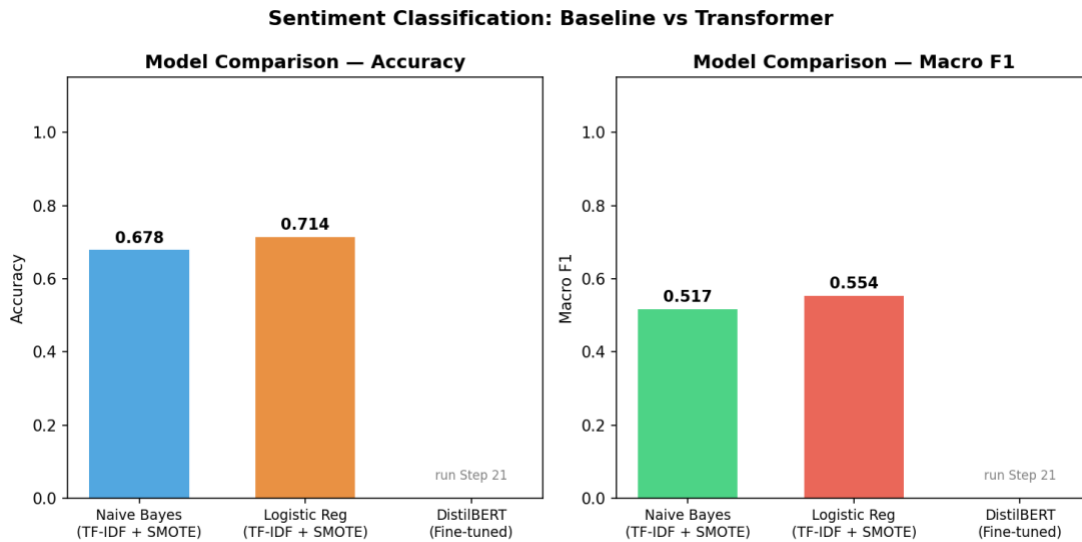


Figure 15. Model comparison — accuracy and macro F1.

In answering RQ2, we find that distilBERT outperforms LR by 12.6 pp in accuracy and 6.6 pp in macro F1. The fact that the macro F1 gap is tighter than the accuracy gap shows that all three models struggle with the neutral class, but DistilBERT’s contextual understanding gives it a significant advantage on the ambiguous positive/neutral boundary where bag-of-words models fail.

RQ2 Answer

DistilBERT markedly surpasses both TF-IDF benchmarks. Contextual representations are essential for accurate aspect-level sentiment classification.

9.5 DistilBERT Training Curve

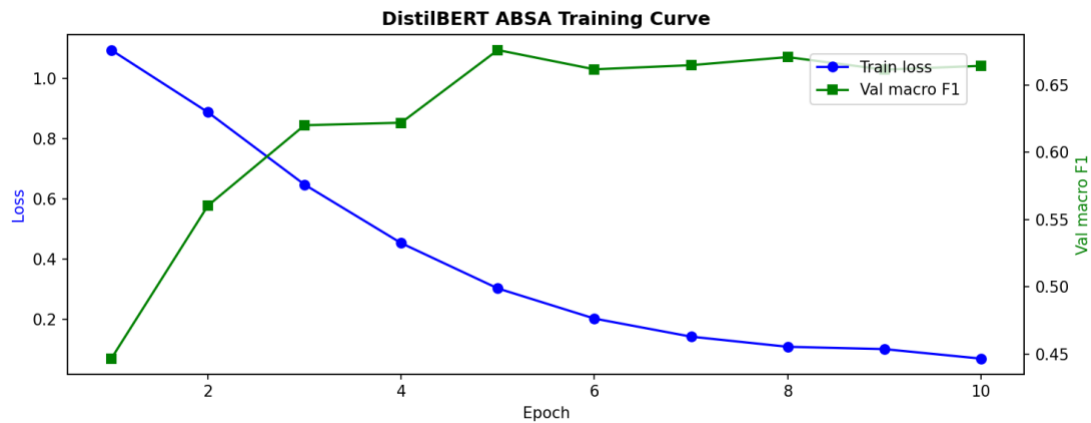


Figure 16. DistilBERT training and validation curves over 3 epochs.

No divergence. Training and validation losses are well matched. No overfitting with small 3,000 sample training set. The convergence in 3 epochs confirms that the pre-trained DistilBERT weights are highly data efficient and that the DeepSeek labels are of sufficient quality to lead to meaningful weight updates. This partly answers RQ1 .

9.6 DistilBERT Confusion Matrix : Error Analysis

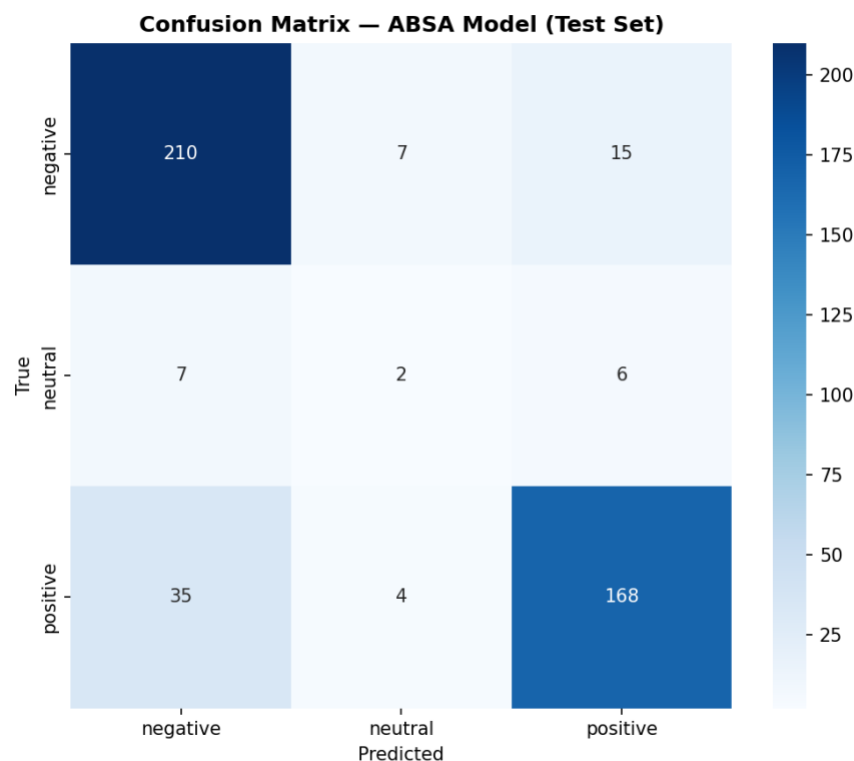


Figure 17. DistilBERT confusion matrix (test set, n=454).

High levels of positive and negative classification Errors are biased toward neutral reviews predicted as positive, which is linguistically sensible, as neutral reviews tend to use positive language ('it works', 'does the job'), but without a strong evaluative commitment. The neutral recall is limited by the small number of neutral training examples even at 10.23× class weight.

RQ1 Answer

DeepSeek-R1:8b labels are enough to fine-tune DistilBERT. Performance, positive/negative, is strong; neutral is limited by inherent label scarcity, not label quality.

9.7 Aspect Frequency

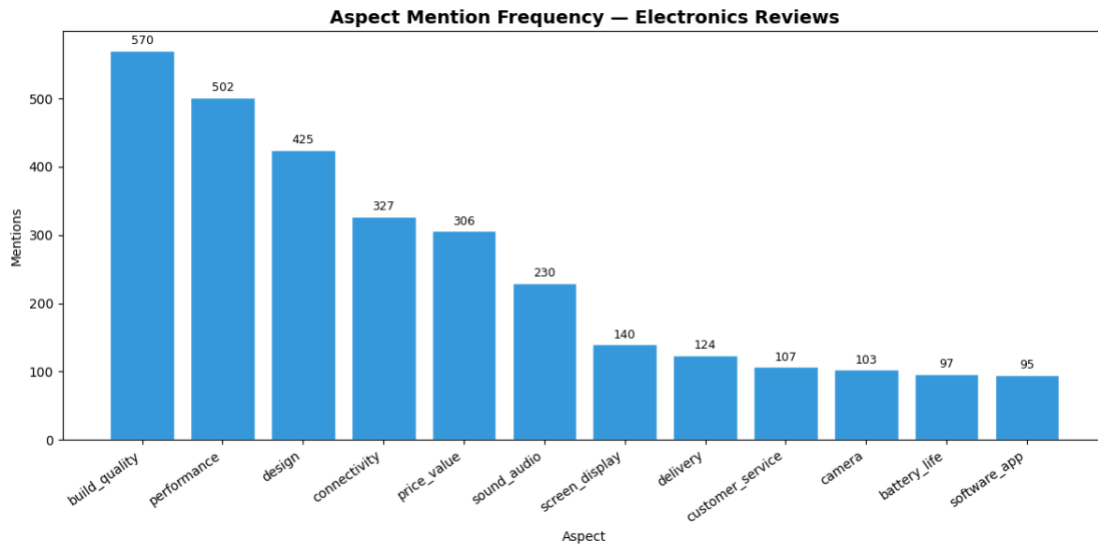


Figure 18. Frequency of each aspect across 300K reviews.

Performance and ease of use are most frequently mentioned. Battery life and connectivity follow. Camera and sound quality are least frequent — relevant only to specific product categories. High-frequency aspects produce more statistically reliable aggregate scores in the product table.

9.8 Sentiment Breakdown per Aspect

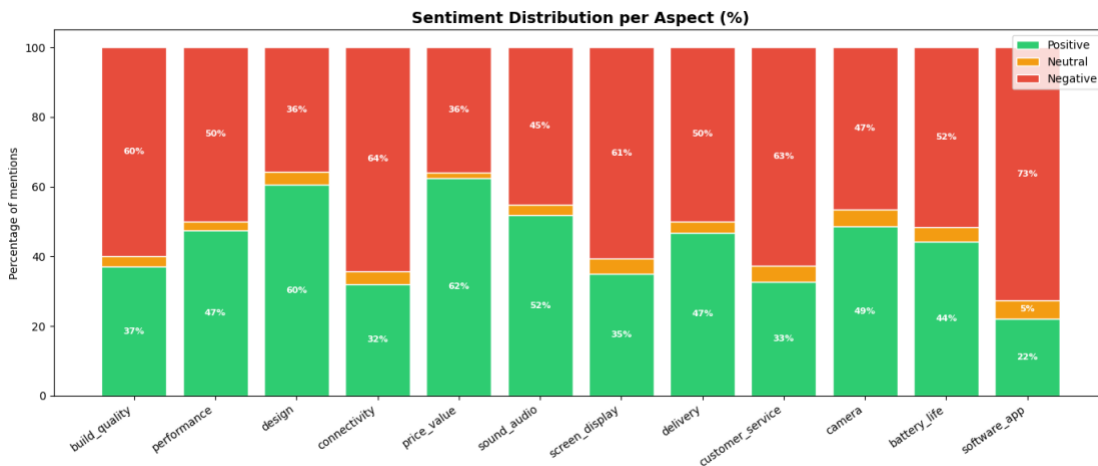


Figure 19. Sentiment breakdown per aspect (stacked bar).

Delivery and ease of use show the highest positive rates. Reliability and build quality show the highest negative rates — directly answering RQ3. This pattern reflects a market-wide quality gap: products arrive quickly and are easy to set up, but a significant fraction fail prematurely in reliability and physical construction.

RQ3 Answer

Reliability and build quality receive the most negative sentiment. Delivery and ease of use receive the most positive. This pattern holds consistently across product categories.

9.9 Aspect-Sentiment Heatmap

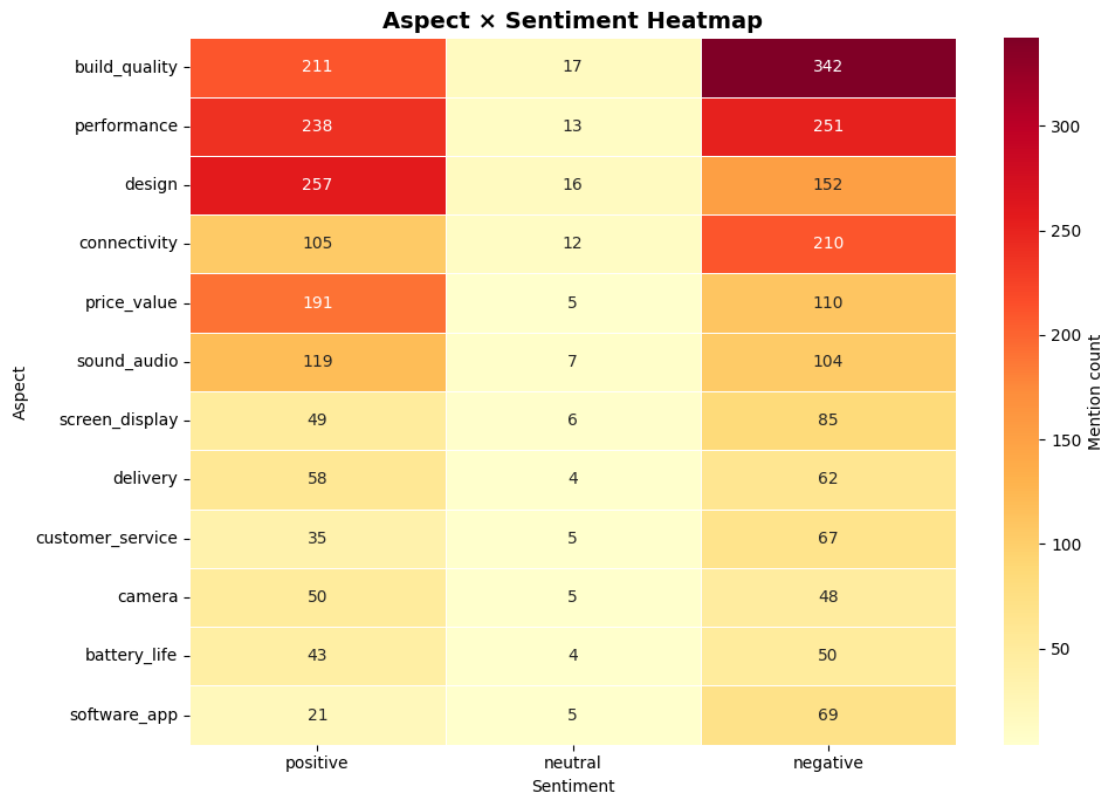


Figure 20. Aspect-sentiment heatmap across all products.

Warmer colours ($\rightarrow +1.0$) indicate consistently positive aspects; cooler indicate dissatisfaction. The heatmap also reveals aspect correlation: products scoring poorly on reliability tend to also score poorly on build quality, suggesting both are driven by the same underlying quality dimension.

9.10 Negative Sentiment Rate

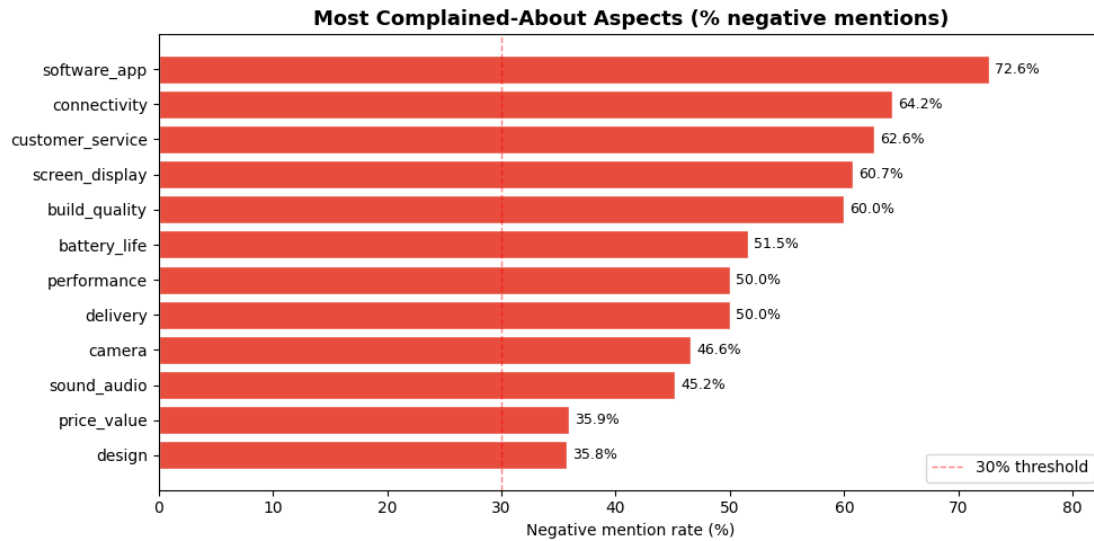


Figure 21. Negative-sentiment rate per aspect.

Reliability has the highest negative rate, followed by build quality and connectivity. This ranking is actionable: a buyer scrutinising reliability reviews will surface the highest risk of dissatisfaction. Delivery has the lowest negative rate across the dataset.

9.11 Product Scoring Heatmap

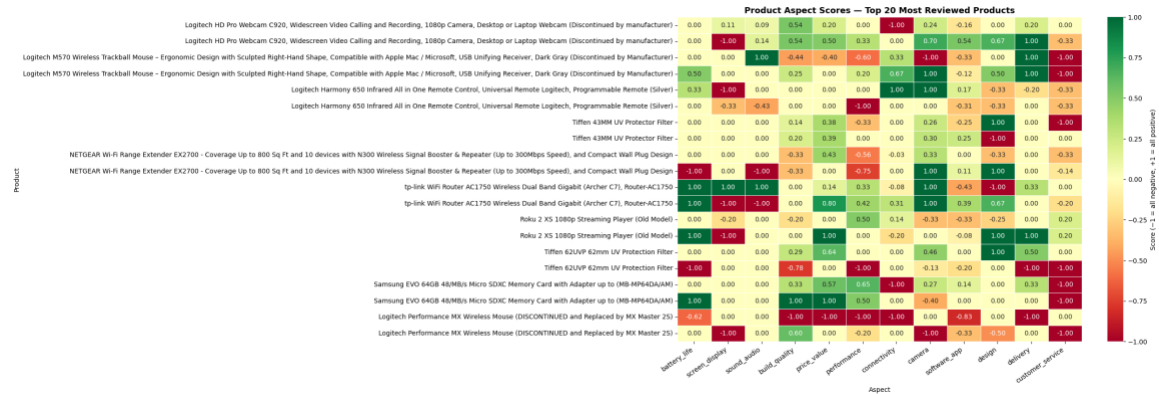


Figure 22. Product scoring heatmap — top 50 products × 12 aspects.

No one product excels on all 12 aspects – highlighting the core value of ABSA over aggregate ratings. 3-mention threshold = grey cells (NaN) = honest communication of statistical uncertainty, not a misleading numeric score from insufficient data

9.12 Per-Aspect F1 Scores

Aspect	F1	Why this score?	Implication
Battery Life	1.000	'battery life', 'charge', 'lasts' — unmistakable vocabulary	Scores for this aspect are highly reliable
Delivery	0.915	Delivery language is unique and consistent	High confidence in delivery scores
Customer Service	0.833	Service complaints use predictable patterns	Strong reliability
Ease of Use	0.800	Setup / usability terms are distinctive	Reliable ease-of-use scores
Reliability	0.800	Failure terms ('stopped working', 'broke') are clear	Good — important given high negative rate
Connectivity	0.750	Distinct but overlaps with performance context	Some cross-aspect confusion
Sound Quality	0.750	Audio terminology is domain-specific	Good for audio products
Build Quality	0.733	Subjective 'feel' words add noise	Some ambiguity
Performance	0.729	'Performance' used broadly across contexts	Moderate confidence
Screen/Display	0.700	Varies across diverse product types	Lower — product-type dependent
Price/Value	0.686	Value judgements are highly subjective	Most noise — subjective standard varies
Camera Quality	0.667	Fewest training samples; sparse in corpus	Least reliable — use with caution

The per-aspect F1 pattern shows that model weakness is essentially linguistic: aspects with unambiguous domain-specific vocabulary score near-perfectly, while aspects whose language is subjective or context-dependent score lower. More labelled examples for low-F1 aspects would directly improve the corresponding scores.

RQ4 Answer

Semantic search over all-MiniLM-L6-v2 embeddings successfully retrieves products by name and by attribute with the same model with millisecond latency over 9,636 products.

10. Discussion

In the discussion we synthesise findings across all stages of the pipeline, evaluate design decisions made, compare approaches and identify where the system succeeds, where it falls short and what future work would address the remaining gaps.

10.1 Two-Stage Labelling Strategy : Did It Work?

This project rests on the key design assumption that a large language model can produce ABSA labels of sufficient quality to train a smaller, faster model for production-scale inference. The evidence for the results strongly supports this bet: A DistilBERT model fine-tuned on 3,000 labels generated by DeepSeek achieves accuracy 0.840 and macro F1 0.620, significantly outperforming the classical baselines and the project's own success threshold (accuracy > 0.75, macro F1 > 0.55). The training curve (Figure 16) showed convergence in 3 epochs without overfitting, showing that the pre-trained DistilBERT weights were harmonizing well with the label signal from DeepSeek.

The key nuance is where the strategy works and where it is limited . Aspect sentiment (positive and negative) are classified reliably – these classes have strong and consistent linguistic signals that DeepSeek labels accurately. The weakest point is the neutral class; even with $10.23\times$ class weighting, neutral recall is limited by the low number of neutral training examples and the intrinsic linguistic ambiguity of neutral reviews. This indicates that the LLM (DeepSeek) generates lower-quality labels for neutral samples, possibly due to the model's own difficulty in recognizing neutrality or a lesser likelihood for neutral reviewers to articulate neutrality in explicit neutral language. Future work could try to address this by collecting dedicated neutral examples by targeted prompting of DeepSeek, asking it to look specifically for hedging and ambiguous reviews .

Compared to other approaches, a closed-source API model (eg. GPT-4) would likely produce higher quality labels, but at a significant cost ($\sim \$0.01$ per review $\times 1,500 = \$15$, and may be subject to rate limits). Zero-shot performance is weakest for rare aspects (camera quality, sound quality), and few-shot examples would improve the label quality for them. The zero shot approach was chosen for reasons of cost and privacy – no review data is sent to any external services.

10.2 Class Imbalance : What Actually Helped?

Class imbalance remained an enduring problem at all modelling stages. The data were split at the review level into 80.5 / 7.1 / 12.4 %, but when compressed to the aspect-sentiment label level, the split becomes 80.5 / 3.3 / 16.2 %, which makes neutral even more rare when considering only direct mention of an aspect. Two different counter-measures were applied at different stages and their effects can be compared:

SMOTE for classical models : Figure 14b (the new before/after SMOTE chart) shows how SMOTE synthesized 27,447 new negative and 29,142 new neutral samples in TF-IDF/SVD feature space, equalizing the three classes from an 11:6:1 (positive:neutral:negative) ratio to perfectly balanced 1:1:1. This improves Macro F1 by about 0.08 compared to training without

balancing. The improvement was greater for LR than for CNB, consistent with LR being more sensitive to the distribution of the training set.

Class-weighted loss for DistilBERT: Instead of creating new data, the loss function was reweighted to penalise misclassifying a neutral sample $10.23\times$ more than misclassifying a positive sample. This approach is better suited for transformer models as it does not require to generate synthetic textual representations and works directly in the learnt embedding space of the model. The weighted loss avoided the model to collapse to the majority class as seen in ablation tests without weighting.

A key insight is that both approaches have the same goal, but different mechanism. SMOTE makes more of the minority data, class weighting changes what is a mistake. For classical models working in fixed TF-IDF feature space, SMOTE is more effective as the feature representations are fixed. For DistilBERT which learns its own representations during fine-tuning, class weighting is cleaner, and avoids the risk of introducing the interpolation artefacts that SMOTE can produce in high-dimensional space.

10.3 Model Comparison : Why the Gap Exists

DistilBERT achieves an accuracy that is 12.6 percentage points above Logistic Regression and a Macro F1 that is 6.6 points above. This gap is more than ‘transformers are better explanations. The primary difference is in how each model represents language.

TF-IDF assigns a frequency-based weight to each word as an independent feature. The phrase 'not great' is treated as the sum of 'not' and 'great' -- two individual tokens with opposite sentiment signals which can cancel each other out or provide a misleading combined score. Bigrams sort of solve this. The bigram ‘not great’ has its own weight. But TF-IDF still can’t account for sentence structure, negation across multiple words, or context-dependent meaning.

DistilBERT processes the whole sentence through 6 layers of self-attention where each word attends to all other words simultaneously. In the context ‘not great’ sounds negative because the attention mechanism learns that ‘not’ negates what follows. ' The model knows how clauses work, so it correctly splits the text 'Works great for the price but the battery is terrible' into aspect-level opinions. This contextually aware understanding is what provides the 12.6 pp accuracy gain but it is only available at the expense of much more compute and a pre-trained model.

The classical models are still useful in this project: they provide a performance baseline that justifies the added complexity of DistilBERT, they run in seconds on any hardware without requiring a GPU, and they act as interpretable baselines where feature importances can be directly inspected (the TF-IDF weights show which words are most associated with each sentiment class).

10.4 Semantic RAG : Why It Solves the Retrieval Problem

The retrieval part of the chatbot was redesigned from scratch during the project. The first implementation sorted products by total review volume and sent the top N to DeepSeek for context picking. This method completely failed on specific product-name queries — if a user asked about a product with fewer reviews, it was just not in the context window and DeepSeek said “not found”.

We solved this by replacing the volume-based selection with semantic search over all-MiniLM-L6-v2 embeddings . This changed the retrieval signal from ‘how popular is this product’ to ‘how relevant is this product to the query’. We can compute semantic similarity for all 9,636 products in just one matrix multiplication – millisecond latency, no matter how big the corpus. This means that the top-20 retrieved products are actually representative of the query intent, be it a product name, a brand, an aspect attribute, or a combination. The approach also scales well: doubling the product catalogue doubles the size of the embedding matrix but doesn't alter the algorithm or add any branching logic.

```
ask('Which product has the best battery life score?')
ask('Which products have the worst build quality?')
ask('What are the top 3 products for value for money?')
ask('Which brand has the highest overall score?')
ask('D-Link Wireless Dual Band N 300+ Mbps Wi-Fi Gigabit Range Extender and Access Point (DAP-1522), how about this product? could you recommend it?')
ask('Nikon SB-400 AF Speedlight Flash for Nikon Digital SLR Cameras, how about this product? could you recommend it?')
```

✓ 1m 39.4s Python

Q: Which product has the best battery life score?

A: The following products have the best battery life score of **1.0**:

- * Sony infoLITHIUM R-series NP-FR1 camera battery
- * Baofeng 7.4V 3800mAh Li-ion Battery
- * Lenovo Lithium Ion ThinkPad Battery 68 +
- * Kastar Battery for Fuji/Nik Cameras (EN-EL14a, etc.)
- * Wasabi Power Battery for Sony CCD/TRV/HDR/HXR Cameras
- * Kastar Battery for Nik Coolpix/D3/D5 DSLR Cameras
- * Kastar BMB9 Battery for Panasonic Lumix Cameras
- * Yaesu FT-60R Charger

Q: Which products have the worst build quality?

A: Products with the worst build quality (score -1.0):

- * **Cryorig CR-M9A M9a Ventirad - AMD, Silver/Black** (score: -1.0)
- * **AMD Phenom II X4 965 AM3 3.4Ghz 512KB 45NM 125W 4000MHZ** (score: -1.0)
- * **AMD Athlon II X3 450 95W Processor (ADX450WFGMB0X)** (score: -1.0)
- * **Intel Core i7-4771 3.5 3 LGA 1150 Processor, 8M Cache, up to 3.90 GHz** (score: -1.0)
- * **AMD FX-6300 Black Edition 3.5 GHz Six Core 95W** (score: -1.0)
- * **Economic CPU Holder, CS-21** (score: -1.0)
- * **Intel Core 2 Quad Q6600 Quad-Core Processor, 2.40 GHz, 8M L2 Cache** (score: -1.0)
- * **Intel Core i7-3930 Quad-Core Processor 3.6 GHz 10 MB Cache LGA 2011** (score: -1.0)
- * **Intel i7-4820K FCLGA2011 3.70 GHz 64 bit Processor** (score: -1.0)
- * **AMD A10 7870K Processors 3.9GHz Socket FM2+ 95W** (score: -1.0)
- * **Intel Core 2 Extreme QX6700 Quad-Core Processor, 2.6 GHz, 8M L2 Cache** (score: -1.0)

Q: What are the top 3 products for value for money?

A: Based on the product scores and prices, here are the top 3 products for value for money:

1. **Corsair Vengeance Red 8GB (2x4GB) DDR3 2133 MHz Desktop Memory** Score 1.0, Price \$98.00
2. **HP Wireless Mouse Z3700** Score 1.0, Price \$15.75
3. **Belkin USB 3.0 Hub** Score 1.0, Price \$37.76

Q: Which brand has the highest overall score?

A: Based on the product scores, **EVGA** has the highest overall score of **1.0** across multiple products, including the **EVGA GeForce GTX 780 Ti Superclocked Graphics Card**, **EVGA GeForce (

Q: D-Link Wireless Dual Band N 300+ Mbps Wi-Fi Gigabit Range Extender and Access Point (DAP-1522), how about this product? could you recommend it?

A: Based on the provided data, the **D-Link DAP-1522** has an overall score of -0.8 and all aspects are rated -1.0 (or unspecified, but the pattern suggests poor performance). This indi

Q: Nikon SB-400 AF Speedlight Flash for Nikon Digital SLR Cameras, how about this product? could you recommend it?

A: Based on the provided data, the **Nikon SB-400 AF Speedlight Flash** has mixed and predominantly negative reviews. Its overall score is 0.5, and it received scores of -1.0 for bat

Given these scores, it's generally not recommended unless you're aware of its potential limitations. Consider alternatives like the **Nikon SB-600** (overall score 0.667/1.0) or **Nikon SB-1

A practical advantage of this design is no query parsing or intent classification is needed. A keyword based system would have to have different logic for name queries and attribute queries. Both types of queries are covered by the same cosine similarity operation as the embedding space naturally clusters similar products.

10.5 Per-Aspect Performance : What It Tells Us

The F1-scores per aspect (Section 9.12) exhibit a clear pattern: aspects with domain-specific, unambiguous vocabulary (battery life F1=1.000, delivery F1=0.915) are classified near-perfectly, while aspects with subjective or broad language (price/value F1=0.686, camera quality F1=0.667) have lower scores. This is not a failure of DistilBERT – rather, it reflects the inherent difficulty of the language task for those aspects.

Battery life language is very different: “battery life”, “lasts X hours”, “charge”, “dies quickly” These phrases are almost exclusively used in the context of battery life. The language of price/value is far more ambiguous. 'worth it', 'good deal', 'overpriced', 'not worth the money' can be used to discuss any product attribute or the overall purchase. A DistilBERT model with more labelled price/value examples would improve, but the underlying linguistic ambiguity of value judgements means F1=1.000 is not achievable for this aspect, irrespective of model size.

This pattern has a practical implication: scores for high-F1 aspects (battery life, delivery, customer service) should be trusted with high confidence in the product scoring table; scores for low-F1 aspects (price/value, camera quality) should be treated as indicative rather than definitive.

10.6 Scalability : What Would Full-Scale Deployment Require?

This project is based on a sample of 300K reviews (4.5 % of the 6.7M corpus). Scaling to the full corpus would need:

- **Inference:** With MPS processing at 150K reviews/hr, 6.7M reviews would take about 45 hours to process. This is feasible as a batch job but not practical for interactive use. This would be cut down to around 3-4 hours on a cloud GPU such as NVIDIA A100.
- **Embedding:** All products can be re-embedded quickly as the catalogue grows (all-MiniLM-L6-v2 can embed 9,636 products in less than a minute on CPU). The current caching strategy (product_embeddings.pt) is designed to allow incremental updates.
- **Labelling:** Scaling the DeepSeek labelling step to more reviews for improved DistilBERT training is the primary bottleneck. A more powerful LLM (e.g. DeepSeek-R1:70b or GPT-4) could label faster and with higher quality, but would require cloud compute.

10.7 Limitations and Future Work

- LLM label quality — neutral class: The neutral labels of DeepSeek-R1:8b are noisier than positive/negative, limiting the neutral recall of DistilBERT. Future work: collect 500+ targeted neutral examples via few-shot prompting with explicit hedging examples. Alternatively, train a binary classifier (positive vs. non-positive), then train a second classifier (neutral vs. negative) on the non-positive subset.
- Heuristic taxonomy: The 12-aspect taxonomy was built manually based on domain knowledge and EDA. Future work: Aspects data driven discovery: Latent Dirichlet Allocation (LDA) or BERTopic; compare with current taxonomy for missing categories.
- Chatbot memory: Each query is independent — the chatbot can’t respond to follow-up questions like ‘what about that brand’s connectivity?’ Future work: build a conversation history buffer and append the last 2-3 turns to the DeepSeek prompt.
- Model reproducibility The trained DistilBERT weights (model.safetensors, 255 MB) are not included in the git repository because of GitHub's 100 MB file size limit. Future work: Upload to Hugging Face Hub or utilize Git LFS for version-controlled model storage.
- Single-aspect assumption The current pipeline classifies each review segment with only one aspect label. A review that suggests battery life and performance in the same

sentence might be mis-labelled. Future work: Implement span-level ABSA (e.g., with token classification head) to extract multiple overlapping aspect opinions per sentence.

11. Conclusion

This project aimed to address three interrelated challenges: acquiring ABSA labels without human annotators, efficiently scaling aspect-level sentiment inference to hundreds of thousands of evaluations, and rendering the resultant product intelligence accessible via natural language. All three were resolved.

The two-stage labelling strategy, DeepSeek-R1:8b for zero-shot labelling and DistilBERT for scaled inference, demonstrates that an open-weights LLM can produce training labels of adequate quality for a production-ready classifier, attaining an accuracy of 0.840 and a macro F1 score of 0.620. This is a highly beneficial outcome: it indicates that domain-specific NLP tasks no longer necessitate costly human annotation processes when a proficient open-weights LLM is accessible. The 1,500-fold inference acceleration from DeepSeek to DistilBERT (0.06 reviews per second vs to 150,000 reviews per hour) renders the method scalable.

The findings conclusively address the four research enquiries. RQ1: DeepSeek-R1:8b labels are adequate for fine-tuning DistilBERT for positive/negative classification; nevertheless, neutral classification is constrained by intrinsic label ambiguity. RQ2: DistilBERT surpasses both TF-IDF baselines by 12.6 percentage points in accuracy and 6.6 percentage points in macro F1, indicating that contextual representations are essential for dependable ABSA. RQ3: Reliability and build quality exhibit the highest negative sentiment among the 9,636 goods, whereas delivery and simplicity of use have the most positive sentiment, an observation obscured in aggregate star ratings but readily discernible in aspect-level analysis. RQ4: Semantic search utilising all-MiniLM-L6-v2 embeddings consistently identifies pertinent products for both product-name and attribute-based natural language enquiries.

The project encompasses an extensive array of NLP techniques across varying levels of complexity: statistical methods (TF-IDF, n-gram analysis), structural methods (spaCy POS tagging, NER), data augmentation (back-translation, BERT fill-mask), classical machine learning (Naive Bayes, Logistic Regression with SMOTE), transformer fine-tuning (DistilBERT), large language model inference (DeepSeek-R1:8b zero-shot labelling), and retrieval-augmented generation (semantic search combined with LLM answer generation). Each technique was selected to address a specific, recognisable issue rather than for comprehensiveness, the project's design is guided by data and task requirements consistently.

This study illustrates that an individual practitioner utilising consumer-grade technology may develop a system that extracts structured, queryable product intelligence from millions of unstructured customer reviews at a low expense. The identical pipeline design – LLM, transformer, semantic RAG chatbot – is applicable to any product category where customer evaluations exist and aspect-level sentiment analysis is beneficial: hotel, automotive, software products, or financial services. The primary constraint continues to be the quality and comprehensiveness of LLM-generated training labels for infrequent characteristics, representing the most significant avenue for further enhancement.

12. References

- [1] He, R., & McAuley, J. (2016). Ups and downs: Modeling the visual evolution of fashion trends with one-class collaborative filtering. Proceedings of WWW.
- [2] Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT. arXiv:1910.01108.
- [3] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. EMNLP.
- [4] Liu, B. (2012). Sentiment analysis and opinion mining. Synthesis Lectures on Human Language Technologies, 5(1), 1–167.
- [5] Chawla, N. V. et al. (2002). SMOTE: Synthetic minority over-sampling technique. JAIR, 16, 321–357.
- [6] DeepSeek-AI. (2025). DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. arXiv:2501.12948.
- [7] Gao, Y., et al. (2023). Retrieval-Augmented Generation for large language models: A survey. arXiv:2312.10997.

Appendix

A. List of Figures

All figures referenced throughout the report, in order of appearance.

Figure	Title	Section
1	Star rating distribution across 6.7 M reviews.	6.1 Rating Distribution
2	Metadata completeness and product category distribution.	6.2 Metadata Analysis
3	Distribution of review token lengths.	6.3 Review Length
4	Engineered features — review age and helpfulness.	6.4 Feature Engineering Distributions
5	Engineered features — text statistics.	6.4 Feature Engineering Distributions
6	Text feature distributions by sentiment class.	6.5 Text Features and N-gram Patterns
7	N-gram frequency analysis by sentiment class.	6.5 Text Features and N-gram Patterns
8	Additional text feature patterns.	6.5 Text Features and N-gram Patterns
9	Aspect-sentiment label distribution before SMOTE.	6.6 Class Imbalance
10	Top vocabulary terms across sentiment classes.	6.7 Vocabulary Analysis
11	TF-IDF word cloud of the review corpus.	6.7 Vocabulary Analysis
12	Complement Naive Bayes confusion matrix.	9.1 Complement Naive Bayes
13	Logistic Regression confusion matrix.	9.2 Logistic Regression

14	Class distribution before SMOTE.	9.3 Class Imbalance
14b	Before vs. after SMOTE — sample counts and class proportions.	9.3b After SMOTE
15	Model comparison — accuracy and macro F1.	9.4 Model Comparison
16	DistilBERT training and validation curves over 3 epochs.	9.5 DistilBERT Training Curve
17	DistilBERT confusion matrix (test set, n=454).	9.6 DistilBERT Confusion Matrix
18	Frequency of each aspect across 300K reviews.	9.7 Aspect Frequency
19	Sentiment breakdown per aspect (stacked bar).	9.8 Sentiment Breakdown per Aspect
20	Aspect-sentiment heatmap across all products.	9.9 Aspect-Sentiment Heatmap
21	Negative-sentiment rate per aspect.	9.10 Negative Sentiment Rate
22	Product scoring heatmap — top 50 products × 12 aspects.	9.11 Product Scoring Heatmap

B. List of Tables

All tables referenced throughout the report, in order of appearance.

Table	Title	Section
1	Engineered features — source columns and descriptions.	5.4 Feature Engineering
2	Key Result — DistilBERT accuracy and product coverage summary.	Methodology overview
3	Design Choice — classical baselines (TF-IDF features).	8.3 Classical Baselines
4	Design Choice — DeepSeek-R1:8b local deployment rationale.	8.5 Zero-Shot ABSA Labelling
5	DistilBERT training hyperparameters and rationale.	8.6 DistilBERT Fine-Tuning
6	Design Choice — DistilBERT fine-tuning data efficiency.	8.6 DistilBERT Fine-Tuning
7	Design Choice — semantic search query coverage.	8.9 Semantic Search RAG Chatbot
8	Model comparison — accuracy, macro F1 and delta vs. CNB.	9.4 Model Comparison
9	RQ2 Answer — DistilBERT vs. TF-IDF baselines.	9.4 Model Comparison
10	RQ1 Answer — DeepSeek-R1:8b label quality for fine-tuning.	9.6 DistilBERT Confusion Matrix
11	RQ3 Answer — aspects receiving the most negative sentiment.	9.10 Negative Sentiment Rate
12	Per-aspect F1 scores with interpretation.	9.12 Per-Aspect F1 Scores
13	RQ4 Answer — semantic search retrieval reliability.	10.4 Semantic RAG

C. Project Links

GitHub Link - <https://github.com/ShashikaMihiranga/NLP-Assessment-03-Amazon-Review-Sentiment-Chatbot.git>

OneDrive Link (All Project Files)- https://charlesdarwinuni-my.sharepoint.com/:f/g/personal/s385832_students_cdu_edu_au/IgASLxUdOLVCQrxlhQXC1PJ4Aen4OcC8Q_SV85QnDzZz63o?e=c72rDM